Informe de Gestión de Vulnerabilidades

Caso: Ferretería y Construcciones del Maule
Ciclo de gestión de vulnerabilidades con Docker como infraestructura: aplicación web Django, base de datos Oracle, ZAP y OpenVAS en la red soc-net

Asignatura	Gestión de Seguridad de la Información (TI3062)
Unidad	Unidad 2 — Ciclo de gestión de vulnerabilidades
Evaluación	Sumativa 2 (ES2) — Docker como infraestructura
Sección	D-IEI-N6-P2-C1
Docente	Héctor Tejo
Estudiantes	Benjamín Cifuentes — Benjamín Contesso — Victor Cancino
Fecha	28 de octubre de 2026

Índice de contenido

1. Introducción 7
2. Objetivos 8
2.1 Objetivo general 8
2.2 Objetivos específicos 8
3. Marco teórico 9
3.1 El ciclo de gestión de vulnerabilidades 9
3.2 Conceptos de Docker 10
Docker y contenedor 10
Imagen (image) 11
Dockerfile 11
Volumen (volume) 11
Redes en Docker 11
Docker Compose 12
3.3 Herramientas de evaluación de vulnerabilidades 12
4. Trabajo previo 13
4.1 Alcance 13
4.2 Funciones y responsabilidades 13
4.3 Herramientas seleccionadas 14
4.4 Inventario de activos 14
5. Construcción de la infraestructura 16
5.1 Red externa soc-net 16
5.2 Contenedor OpenVAS 17
Incidente: reinicios en bucle por la sincronización de feeds 18
5.3 Contenedor Oracle 19
Incidente: el usuario de aplicación no se creó 20
5.4 Aplicación web con Dockerfile 22
5.5 Contenedor ZAP 28
5.6 Evidencia de la red: contenedores e IP 30
5.7 Diagramas de los archivos Compose 31
6. Identificación: escaneos de vulnerabilidades 35
6.1 ZAP — Escaneo 1: exploración y análisis pasivo 35
6.2 ZAP — Escaneo 2: escaneo activo 37
6.3 OpenVAS — Pasada 1 40
6.4 OpenVAS — Pasada 2 43

6.5 Revisión de configuración 46
7. Evaluación y priorización: tabla de resumen de vulnerabilidades 47
8. Actuar: estrategia de mitigación 49
9. Verificar y mejorar 51
9.1 Aplicación de correcciones y escaneo de verificación 51
9.2 Métricas 53
9.3 Mejora continua 54
10. Conclusiones 55
Referencias 57
Anexos 59
Anexo A. Archivos docker-compose.yml 59
A.1 Oracle (01-oracle/docker-compose.yml, entregado por el docente) 59
A.2 Aplicación web (02-app-django/docker-compose.yml, versión final con la imagen 1.1) 59
A.3 ZAP (03-zap/docker-compose.yml) 59
A.4 OpenVAS (04-openvas/docker-compose.yml, versión final con SKIPSYNC) 60
Anexo B. Dockerfile de la aplicación web 61
Anexo C. Comandos de la demostración 61

Índice de figuras

Figura 1. Ciclo de gestión de vulnerabilidades. Tomado del enunciado de la Evaluación 2 (U2ES2). 9

Figura 2. Arquitectura solicitada: cuatro contenedores en la red soc-net. Tomado del enunciado (U2ES2). 13

Figura 3. Redes Docker existentes antes de comenzar (`docker network ls`). 16

Figura 4. Creación de la red externa soc-net con la subred 172.28.0.0/16. 16

Figura 5. Inspección de soc-net: controlador bridge, subred 172.28.0.0/16 y gateway 172.28.0.1. 16

Figura 6. Archivo docker-compose.yml de OpenVAS (versión inicial). 17

Figura 7. Levantamiento de OpenVAS: se crea el volumen y se inicia el contenedor. 17

Figura 8. Registros de arranque de OpenVAS: certificados y restauración de la base de datos.. 18

Figura 9. Causa de los reinicios: la descarga de feeds por rsync se corta y el contenedor se detiene. 19

Figura 10. Corrección: variable SKIPSYNC en el compose y recreación del contenedor. 19

Figura 11. Archivo docker-compose.yml de Oracle entregado por el docente (sin cambios). 20

Figura 12. Levantamiento de Oracle con docker compose up -d. 20

Figura 13. Registros de Oracle: «DATABASE IS READY TO USE!». 20

Figura 14. Primer intento de crear las tablas: error ORA-01017 (credenciales inválidas). 21

Figura 15. Causa: ORA-65012 en el primer arranque, reinicio del contenedor y omisión del usuario de aplicación. 21

Figura 16. Creación del usuario de aplicación con createAppUser y verificación en DBA_USERS. 21

Figura 17. Creación de las tablas CATEGORIA y PRODUCTO e inserción de dos registros en cada una. 22

Figura 18. Consulta de las tablas y unión de producto con su categoría. 22

Figura 19. Estructura del proyecto Django (carpeta 02-app-django). 23

Figura 20. Dockerfile comentado de la aplicación web. 24

Figura 21. Construcción de la imagen ferreteria-web:1.0 (12 de 12 pasos). 24

Figura 22. Imágenes disponibles: la imagen propia y las descargadas. 25

Figura 23. Archivo docker-compose.yml de la aplicación web. 25

Figura 24. Levantamiento del contenedor app-web. 25

Figura 25. Registros de app-web: Oracle disponible, migraciones, superusuario y gunicorn. 26

Figura 26. Catálogo de productos leído desde Oracle (http://localhost:8088). 26

Figura 27. Búsqueda de productos por nombre. 27

Figura 28. Formulario de ingreso al panel de administración (/admin/). 27

Figura 29. Panel de administración con los modelos del catálogo. 28
Figura 30. Productos de Oracle administrados desde el panel de Django. 28
Figura 31. Archivo docker-compose.yml de ZAP. 29
Figura 32. Levantamiento del contenedor ZAP. 29
Figura 33. Interfaz de ZAP 2.17.0 en el navegador (Webswing). 30
Figura 34. Los cuatro contenedores en ejecución (docker ps). 30
Figura 35. Contenedores de soc-net con su dirección IP (comando del docente). 30
Figura 36. Volúmenes persistentes de Oracle y OpenVAS. 31
Figura 37. Diagrama de bloques del compose de Oracle (YAMLTools). 31
Figura 38. Diagrama de bloques del compose de la aplicación web (YAMLTools). 32
Figura 39. Diagrama de bloques del compose de ZAP (YAMLTools). 33
Figura 40. Diagrama de bloques del compose de OpenVAS (YAMLTools). 34
Figura 41. Escaneo 1: árbol del sitio descubierto por el spider de ZAP. 35
Figura 42. Escaneo 1: alertas de ZAP y detalle de «Content Security Policy (CSP) Header Not Set». 36
Figura 43. Reporte 1 de ZAP en HTML: resumen de alertas por nivel de riesgo. 36
Figura 44. Escaneo 2: escaneo activo en curso (35 %). 37
Figura 45. Escaneo 2: escaneo activo terminado (3 444 peticiones). 38
Figura 46. Escaneo 2: alerta «HTTP Only Site» detectada por el escaneo activo. 38
Figura 47. Reporte 2 de ZAP en HTML. 39
Figura 48. Reportes de ZAP generados en formato JSON y HTML. 39
Figura 49. Ingreso a la interfaz web de OpenVAS (GSA). Se observa la advertencia de conexión sin cifrar. 40
Figura 50. Panel de OpenVAS con las NVT cargadas por severidad. 41
Figura 51. Objetivo creado: contenedor de la aplicación web (172.28.0.20). 41
Figura 52. Tarea «Full and fast» creada sobre el objetivo. 42
Figura 53. Pasada 1 terminada: duración, estado y cantidad de resultados. 42
Figura 54. Pasada 1: hallazgos de severidad Baja. 43
Figura 55. Tarea con dos reportes (dos pasadas), estado Done y severidad 2,6 (Baja). 44
Figura 56. Reportes de ambas pasadas con el conteo por severidad. 44
Figura 57. Pasada 2: mismos hallazgos de severidad Baja. 45
Figura 58. Reportes de OpenVAS exportados (XML, CSV y PDF) por pasada. 45
Figura 59. Construcción de la versión 1.1 con oracledb 4.0.2. 51
Figura 60. Aplicación 1.1 en ejecución: CSP presente y estáticos sin CORS permisivo. 51
Figura 61. Escaneo de verificación: cabecera CSP en la respuesta y alertas restantes. 52
Figura 62. Reporte 3 de ZAP (verificación tras mitigar). 53

Índice de tablas

Tabla 1 *Fases del ciclo de gestión de vulnerabilidades aplicadas en la evaluación* 10
Tabla 2 *Funciones definidas para el proceso* 13
Tabla 3 *Herramientas y versiones utilizadas* 14
Tabla 4 *Inventario de activos de la infraestructura* 14
Tabla 5 *Resumen de alertas de ZAP por escaneo* 39
Tabla 6 *Comparación de las dos pasadas de OpenVAS* 45
Tabla 7 *Resumen de vulnerabilidades encontradas* 47
Tabla 8 *Cantidad de hallazgos por nivel de riesgo* 48
Tabla 9 *Estrategia de mitigación priorizada* 49
Tabla 10 *Comparación antes y después de las correcciones* 53
Tabla 11 *Métricas de los escaneos realizados* 53

1. Introducción

El presente informe documentó la aplicación del **ciclo de gestión de vulnerabilidades** sobre una infraestructura construida completamente con contenedores Docker. El ciclo, que el docente presentó como basado en el marco del NIST, propone identificar las vulnerabilidades de una infraestructura, evaluarlas y priorizarlas según su impacto, actuar sobre ellas (corregir, mitigar o aceptar el riesgo), verificar que las correcciones funcionaron y mejorar el proceso de forma continua (Souppaya y Scarfone, 2022; CrowdStrike, s.f.).

Como caso se retomó la empresa *Ferretería y Construcciones del Maule*, analizada en la Evaluación 1. Se desarrolló una aplicación web propia en Django que publica el catálogo de productos de la ferretería y lo lee desde una base de datos Oracle. Ambos componentes, junto con dos escáneres de seguridad —ZAP para aplicaciones web y OpenVAS para infraestructura—, se desplegaron como cuatro contenedores conectados a una red Docker externa llamada `soc-net`. Cada contenedor quedó definido en su propio archivo `docker-compose.yml`, de modo que su configuración queda guardada y se puede recrear sin depender de comandos sueltos.

El documento presenta primero el marco teórico (el ciclo y los conceptos de Docker solicitados) y el trabajo previo (alcance, herramientas e inventario de activos). Luego relata paso a paso la construcción de la infraestructura, incluidos los dos problemas reales que aparecieron y cómo se resolvieron, los cuatro escaneos exigidos (dos con ZAP y dos pasadas de OpenVAS), la tabla de resumen de vulnerabilidades y la estrategia de mitigación priorizada. Finalmente se aplicaron tres correcciones y se volvió a escanear la aplicación, cerrando una vuelta completa del ciclo. Todo el proceso se respaldó con capturas tomadas durante la ejecución real.

2. Objetivos

2.1 Objetivo general

Aplicar el ciclo de gestión de vulnerabilidades a una infraestructura de contenedores Docker compuesta por una aplicación web propia, una base de datos Oracle y dos escáneres de seguridad, documentando el proceso con evidencias que permitan repetirlo.

2.2 Objetivos específicos

- Construir la imagen de la aplicación web a partir de un Dockerfile y desplegar los cuatro contenedores en la red externa `soc-net`, cada uno con su archivo `docker-compose.yml`.
- Identificar vulnerabilidades en la aplicación web mediante dos escaneos con ZAP y en su contenedor mediante dos pasadas con OpenVAS.
- Evaluar y priorizar los hallazgos en una tabla de resumen que indique el activo afectado, el impacto y el riesgo según la categoría de cada herramienta.
- Proponer una estrategia de mitigación ordenada por prioridad y verificar con un nuevo escaneo el efecto de las correcciones aplicadas.

3. Marco teórico

3.1 El ciclo de gestión de vulnerabilidades

La gestión de vulnerabilidades no es una revisión única, sino un proceso **cíclico**: una aplicación que hoy no presenta vulnerabilidades puede tenerlas mañana, por ejemplo, cuando se publica una falla en la versión de Python o de una librería que utiliza. La guía SP 800-40 del NIST plantea la gestión de parches y vulnerabilidades como mantenimiento preventivo permanente (Souppaya y Scarfone, 2022). La Figura 1 muestra el ciclo entregado en el enunciado, que coincide con el ciclo de vida descrito por CrowdStrike (s.f.): un trabajo previo y cinco fases que se repiten.



Figura 1. Ciclo de gestión de vulnerabilidades. Tomado del enunciado de la Evaluación 2 (U2ES2).

La Tabla 1 resume cómo se aplicó cada fase en este trabajo.

Tabla 1
Fases del ciclo de gestión de vulnerabilidades aplicadas en la evaluación

Fase	Actividades (según la figura)	Aplicación en este trabajo	Sección
Trabajo previo	Determinar el alcance; definir funciones y responsabilidades; seleccionar herramientas; preparar el inventario de activos	Alcance: 4 contenedores en `soc-net`. Herramientas: ZAP y OpenVAS. Inventario de activos con IP y puertos	4
Identificar	Identificar; escanear	Construcción de la infraestructura; 2 escaneos con ZAP, 2 pasadas con OpenVAS y revisión de configuración	5 y 6
Evaluar y priorizar	Agregar contexto de amenaza; asignar valor	Tabla de resumen con activo, impacto y riesgo de cada hallazgo	7
Actuar	Corregir; mitigar; aceptar el riesgo	Estrategia de mitigación priorizada	8
Verificar	Validar; volver a escanear	Aplicación de 3 correcciones (app 1.1) y escaneo de verificación con ZAP	9
Mejorar	Evaluar métricas; automatizar procesos	Métricas de los escaneos y propuesta de escaneos periódicos	9

3.2 Conceptos de Docker

Los siguientes conceptos, solicitados en el enunciado, se definen a partir de la documentación oficial de Docker.

Docker y contenedor

Docker es una plataforma abierta para desarrollar, distribuir y ejecutar aplicaciones dentro de **contenedores**. Un contenedor es una instancia en ejecución de una imagen: un proceso aislado que incluye todo lo que la aplicación necesita (código, intérprete, librerías y configuración) y que comparte el núcleo del sistema anfitrión. Esto permite separar la aplicación de la infraestructura y ejecutar en un mismo equipo, por ejemplo, dos aplicaciones con versiones distintas de Python sin que sus librerías se mezclen (Docker Inc., s.f.-a).

Imagen (image)

Una imagen es una plantilla de solo lectura con las instrucciones para crear un contenedor. Está formada por capas, se identifica con un nombre y una etiqueta (`nombre:tag`, donde la etiqueta suele indicar la versión) y se obtiene desde un registro como Docker Hub o se construye a partir de un Dockerfile (Docker Inc., s.f.-a). En este trabajo, Oracle, ZAP y OpenVAS usan imágenes publicadas; solo la imagen de la aplicación web (`ferreteria-web`) es propia.

Dockerfile

Un Dockerfile es un archivo de texto que contiene, en orden, las instrucciones (`FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, `CMD`, entre otras) que el comando `docker build` ejecuta para construir una imagen (Docker Inc., s.f.-d). El detalle de cada instrucción se desarrolla en el documento complementario `Dockerfile.md`.

Volumen (volume)

Un volumen es un mecanismo de almacenamiento persistente administrado por Docker e independiente del ciclo de vida del contenedor: los datos se conservan aunque el contenedor se elimine o se vuelva a crear (Docker Inc., s.f.-c). Se usaron los volúmenes `oracle-volume` (datos de Oracle) y `openvas-data` (base de datos, feeds y resultados de OpenVAS).

Redes en Docker

Las redes permiten que los contenedores se comuniquen entre sí y con el exterior. El controlador por defecto es `bridge`; una red *bridge* definida por el usuario, como `soc-net`, agrupa a los contenedores, los aísla de otros y permite que se encuentren por nombre gracias al DNS interno de Docker (Docker Inc., s.f.-b). Dos contenedores en redes distintas no se

comunican, lo que constituye una primera estrategia de seguridad. En Compose, `external: true` indica que la red ya existe y fue creada antes, fuera del archivo.

Docker Compose

Docker Compose es la herramienta para definir y ejecutar aplicaciones de uno o varios contenedores mediante un archivo YAML (`docker-compose.yml`) que declara servicios, imagen, puertos, redes, volúmenes y variables. Se levanta con `docker compose up -d` (Docker Inc., s.f.-e). Su ventaja frente a un comando suelto es que la configuración queda guardada en un archivo versionable, por lo que el contenedor se puede recrear siempre igual.

3.3 Herramientas de evaluación de vulnerabilidades

ZAP (Zed Attack Proxy) es un escáner de seguridad para aplicaciones web de código abierto. Explora el sitio con su *spider*, analiza pasivamente cada respuesta y, en el escaneo activo, envía peticiones de ataque (inyección SQL, XSS, entre otras) para confirmar vulnerabilidades. Clasifica sus alertas en Alto, Medio, Bajo e Informativo. Se usó su imagen oficial con la interfaz gráfica completa servida en el navegador mediante Webswing (ZAP, s.f.-a; ZAP, s.f.-b).

OpenVAS, parte de Greenbone Community Edition, es un escáner de vulnerabilidades de infraestructura de código abierto, alternativa libre a Nessus. Ejecuta decenas de miles de pruebas (NVT) contra los servicios de un equipo y asigna a cada hallazgo una severidad CVSS (Greenbone AG, s.f.). Se usó la imagen todo-en-uno de Immauss, que incluye el escáner, el gestor y la interfaz web GSA (Immauss, s.f.).

4. Trabajo previo

4.1 Alcance

El alcance de la evaluación son los cuatro contenedores definidos en el enunciado y conectados a la red `soc-net` (Figura 2). Los objetivos de los escaneos son la aplicación web (ZAP) y el contenedor que la aloja (OpenVAS). Además, se revisó la configuración de los cuatro archivos `docker-compose.yml` en busca de malas prácticas.

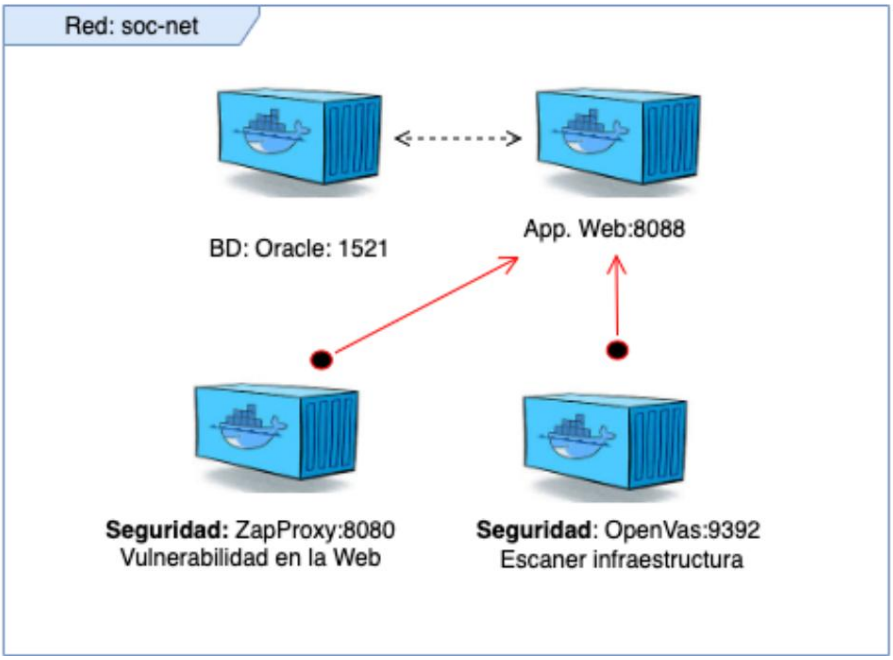


Figura 2. Arquitectura solicitada: cuatro contenedores en la red soc-net. Tomado del enunciado (U2ES2).

4.2 Funciones y responsabilidades

Tabla 2
Funciones definidas para el proceso

Función	Responsabilidad
Responsable de infraestructura	Crear la red, escribir los archivos Compose y el Dockerfile, construir la imagen y mantener los contenedores en ejecución
Analista de vulnerabilidades	Configurar ZAP y OpenVAS, ejecutar los escaneos y exportar los reportes
Responsable de remediación	Analizar los hallazgos, priorizarlos, aplicar las correcciones y solicitar el escaneo de verificación

Función	Responsabilidad
Documentación	Registrar cada paso con capturas, redactar el informe y preparar la demostración

4.3 Herramientas seleccionadas

Tabla 3
Herramientas y versiones utilizadas

Herramienta	Versión	Uso
Docker Desktop / Docker Engine	Engine 29.7.2, Compose v5.3.1	Plataforma de contenedores (Windows 11 + WSL 2)
Oracle Database Free	gvenzl/oracle-free:latest (26ai)	Base de datos de la aplicación
Python / Django	python:3.12-slim / Django 5.2.18 LTS	Aplicación web (catálogo de la ferretería)
ZAP	zaproxy/zap-stable 2.17.0	Escáner de aplicaciones web
OpenVAS (Greenbone)	immauss/openvas 26.07.27.01 (GSA 27.5.0, gvmd 26.34.0)	Escáner de infraestructura
YAMLTools Visualizer	yamltools.dev (web)	Diagramas de los archivos Compose
Asistente de IA (Claude)	—	Apoyo para generar y explicar archivos, resumir reportes y redactar; el grupo revisó cada resultado

4.4 Inventario de activos

La Tabla 4 presenta el inventario de activos. Las direcciones IP se obtuvieron con el comando de red que se muestra en la sección 5.6.

Tabla 4
Inventario de activos de la infraestructura

Activo	Imagen	IP en soc-net	Puertos (host:contenedor)	Función	Criticidad
oracle-db	gvenzl/oracle-free:latest	172.28.0.3	1521:1521	Base de datos (tablas CATEGORIA y PRODUCTO)	Alta
app-web	ferreteria-web:1.0 → 1.1	172.28.0.20 (fija)	8088:8000	Catálogo web y panel de administración	Alta
zap	zaproxy/zap-stable:latest	172.28.0.4	8080:8080, 8090:8090	Escáner web (interfaz y API)	Media
openvas	immauss/openvas:latest	172.28.0.2	9392:9392	Escáner de infraestructura	Media
soc-net	Red bridge externa	172.28.0.0/16 (gw .1)	—	Comunicación entre contenedores	Alta

Activo	Imagen	IP en soc-net	Puertos (host:contenedor)	Función	Criticidad
Volúmenes	oracle-volume, openvas-data	—	—	Persistencia de datos	Alta

5. Construcción de la infraestructura

Esta sección relata, en el orden en que se ejecutó el 27 de octubre de 2026, cómo se construyó la infraestructura. OpenVAS se levantó en segundo lugar, inmediatamente después de la red, porque al iniciar debe cargar su base de pruebas de vulnerabilidades y el docente advirtió que puede tardar entre 10 minutos y una hora. Todos los archivos se encuentran en las carpetas `01-oracle`, `02-app-django`, `03-zap` y `04-openvas`.

5.1 Red externa soc-net

Primero se revisaron las redes existentes (Figura 3). La red `soc-net` existía de una prueba anterior con otra subred, por lo que se eliminó y se volvió a crear con la subred `172.28.0.0/16` indicada en el archivo de configuración del curso (Figuras 4 y 5). Al ser externa, debe existir antes de levantar cualquier contenedor.



Figura 3. Redes Docker existentes antes de comenzar (`docker network ls`).

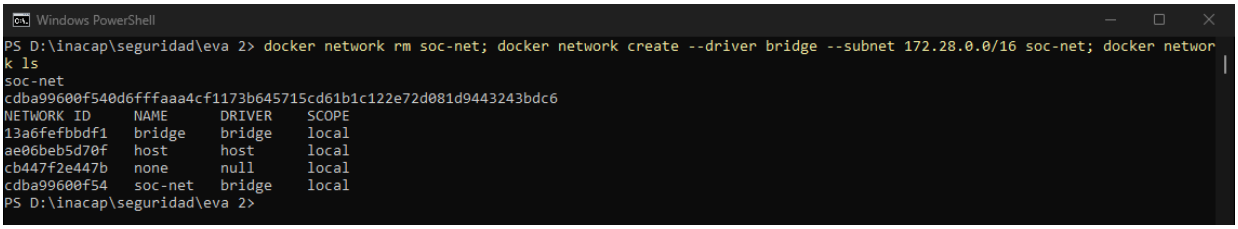


Figura 4. Creación de la red externa soc-net con la subred 172.28.0.0/16.

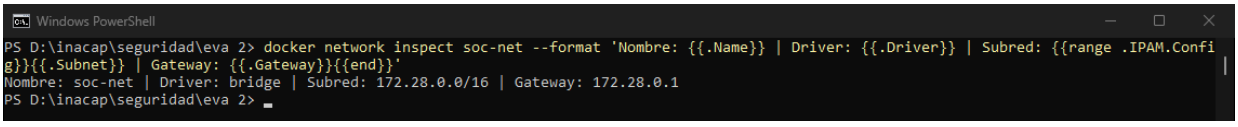


Figura 5. Inspección de soc-net: controlador bridge, subred 172.28.0.0/16 y gateway 172.28.0.1.

5.2 Contenedor OpenVAS

El archivo `04-openvas/docker-compose.yml` (Figura 6) usa la imagen `immauss/openvas`, publica la interfaz web en el puerto 9392, guarda los datos en el volumen `openvas-data` y conecta el contenedor a `soc-net`. La contraseña del usuario `admin` no se escribe en el archivo: se lee de la variable `OPENVAS_PASSWORD` definida en un archivo `.env` que no se entrega. El contenedor se levantó con `docker compose up -d` (Figura 7) y en sus registros se observa la restauración de la base de datos inicial (Figura 8).

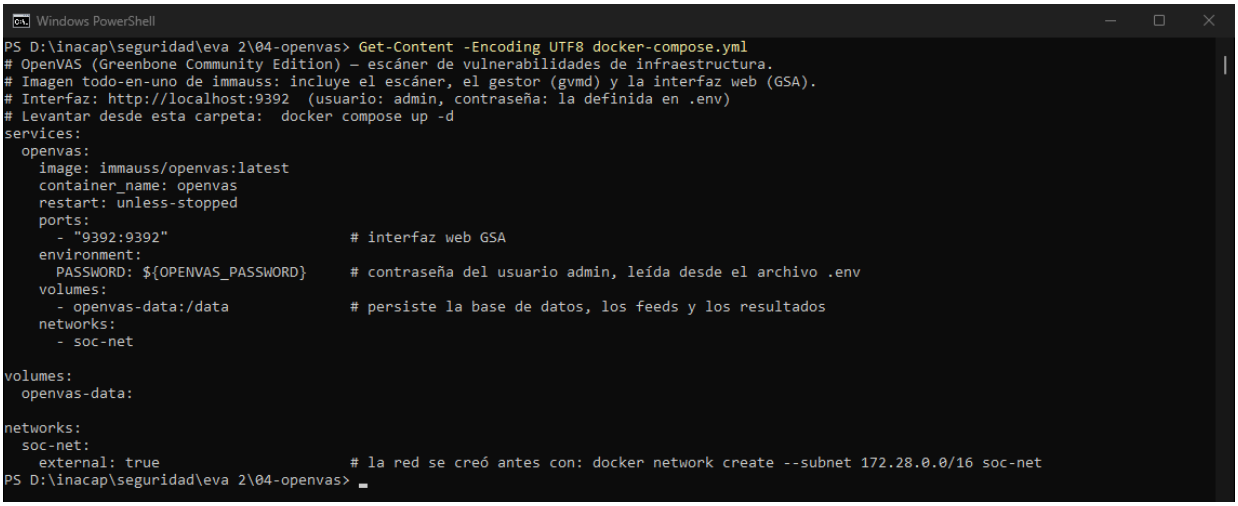


Figura 6. Archivo docker-compose.yml de OpenVAS (versión inicial).

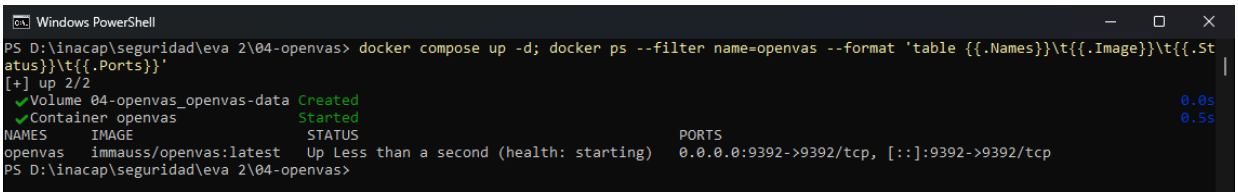


Figura 7. Levantamiento de OpenVAS: se crea el volumen y se inicia el contenedor.

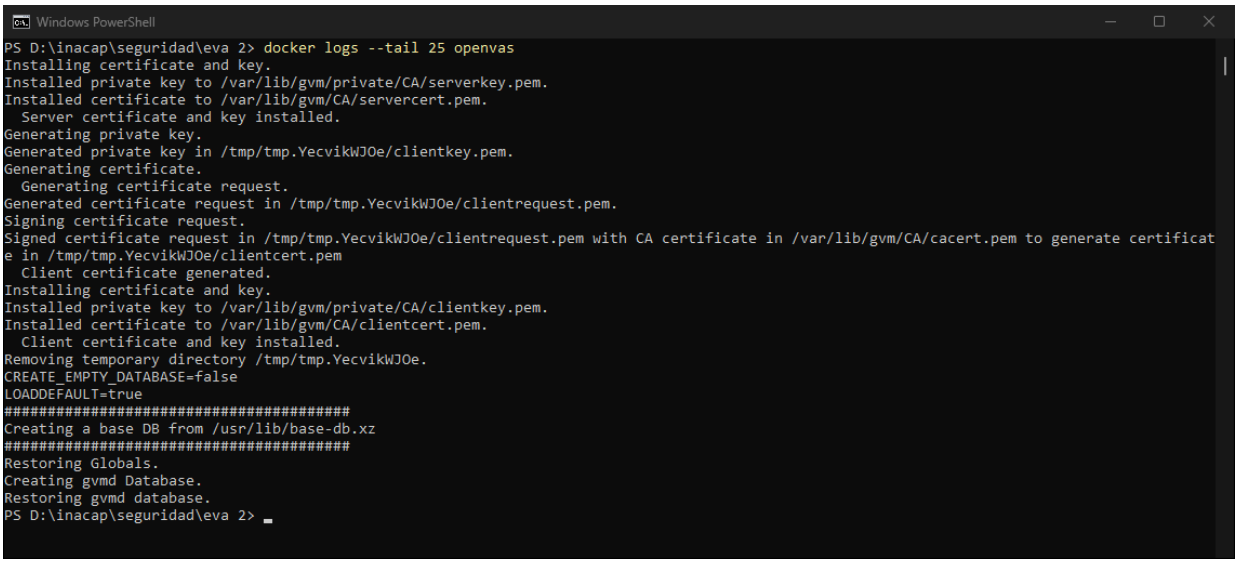


Figura 8. Registros de arranque de OpenVAS: certificados y restauración de la base de datos.

Incidente: reinicios en bucle por la sincronización de feeds

A los 25 minutos el gestor de OpenVAS aún no respondía y el contenedor llevaba dos reinicios. Al revisar los registros (Figura 9) se encontró la causa: en cada arranque, el script de la imagen intenta descargar los feeds desde el servidor público de Greenbone mediante `rsync`; la conexión se cortaba (`rsync error ... code 12`) y, según el propio script, si la sincronización falla el contenedor termina, Docker lo reinicia y el ciclo se repite. La solución fue agregar la variable `SKIPSYNC: "true"`, documentada por la imagen, para usar los feeds incluidos en ella (fechados el 6 de octubre de 2026), y recrear el contenedor conservando el volumen (Figura 10). Tres minutos después OpenVAS quedó operativo, con la configuración de escaneo «Full and fast» y más de 192 000 pruebas (NVT) cargadas. Esta decisión deja un riesgo que se registra como hallazgo V14: los feeds no incluyen vulnerabilidades publicadas después de esa fecha.

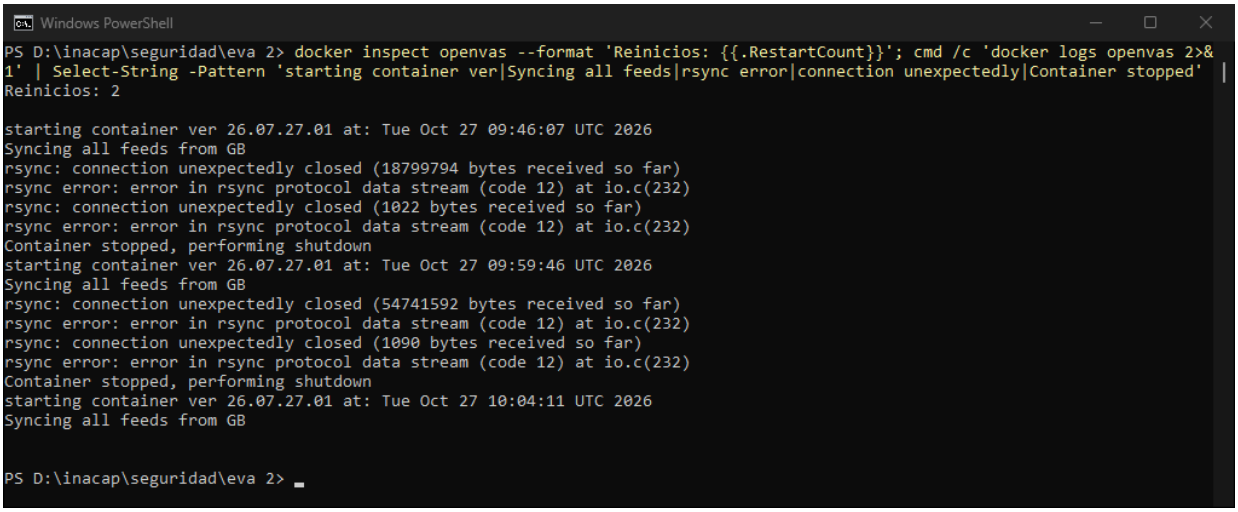


Figura 9. Causa de los reinicios: la descarga de feeds por rsync se corta y el contenedor se detiene.

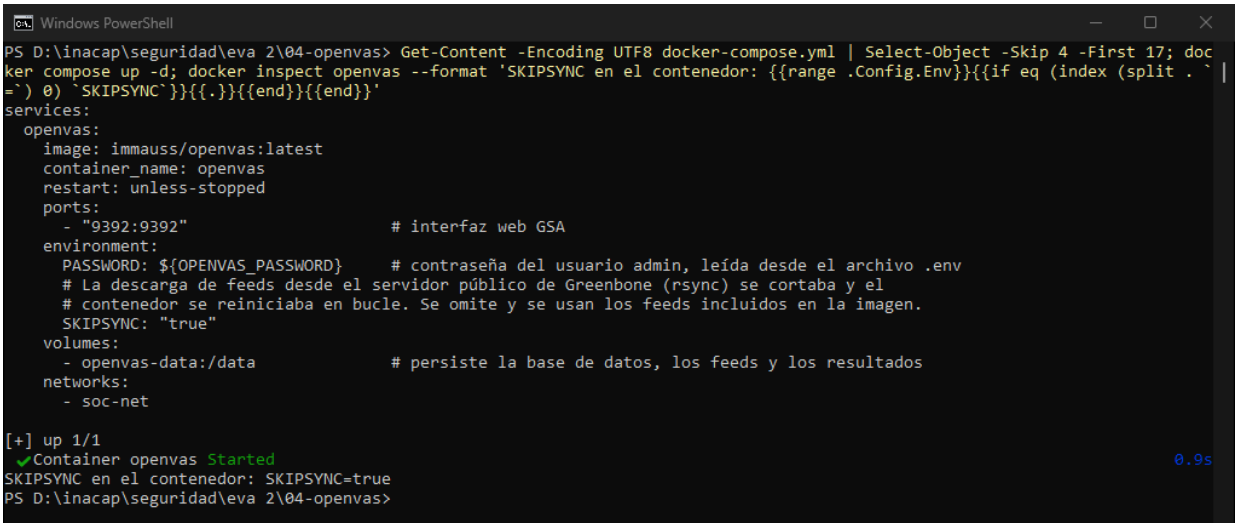


Figura 10. Corrección: variable SKIPSYNC en el compose y recreación del contenedor.

5.3 Contenedor Oracle

Para Oracle se utilizó, sin modificaciones, el archivo `docker-compose.yml` entregado por el docente en el material del curso (Figura 11). Este archivo contiene las contraseñas escritas directamente en las variables de entorno; el docente explicó en clase que lo dejó así a propósito, como mala práctica, para ver si los escáneres la detectan. Al levantarlo, Docker advierte que el atributo `version` está obsoleto (Figura 12), lo que coincide con lo señalado en clase: esa línea es opcional. La base de datos quedó lista en 135 segundos (Figura 13).

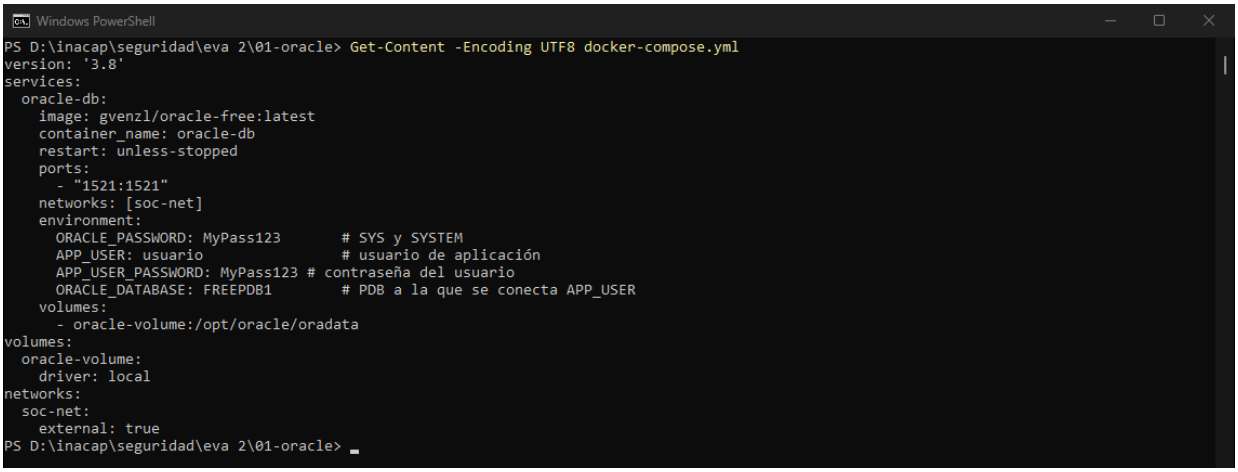


Figura 11. Archivo docker-compose.yml de Oracle entregado por el docente (sin cambios).

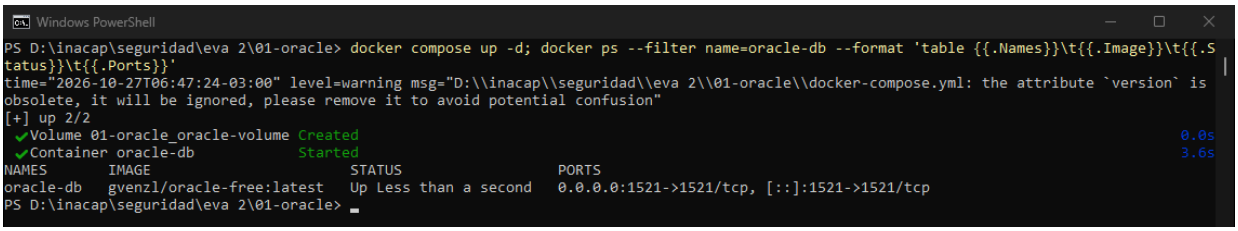


Figura 12. Levantamiento de Oracle con docker compose up -d.

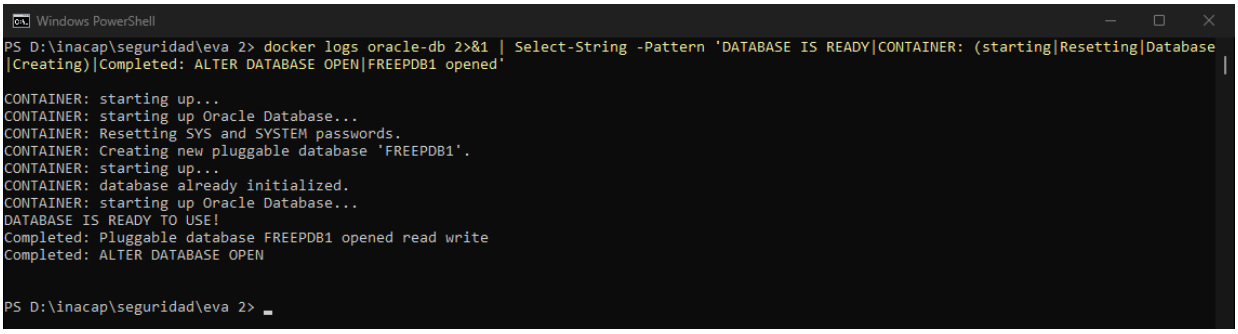


Figura 13. Registros de Oracle: «DATABASE IS READY TO USE!».

Incidente: el usuario de aplicación no se creó

Al intentar crear las tablas con el usuario `usuario`, Oracle rechazó la conexión con el error `ORA-01017` (Figura 14). Los registros mostraron la causa (Figura 15): el compose pide crear la base de datos acoplable `FREEPDB1` (`ORACLE_DATABASE: FREEPDB1`), pero la versión actual de la imagen ya la incluye; el script de inicio falló con `ORA-65012`, el contenedor se reinició y en el segundo arranque se consideró ya inicializado, por lo que nunca creó el usuario

de aplicación. Sin modificar el archivo del docente, se creó el usuario con el script `createAppUser` que trae la imagen (Figura 16).

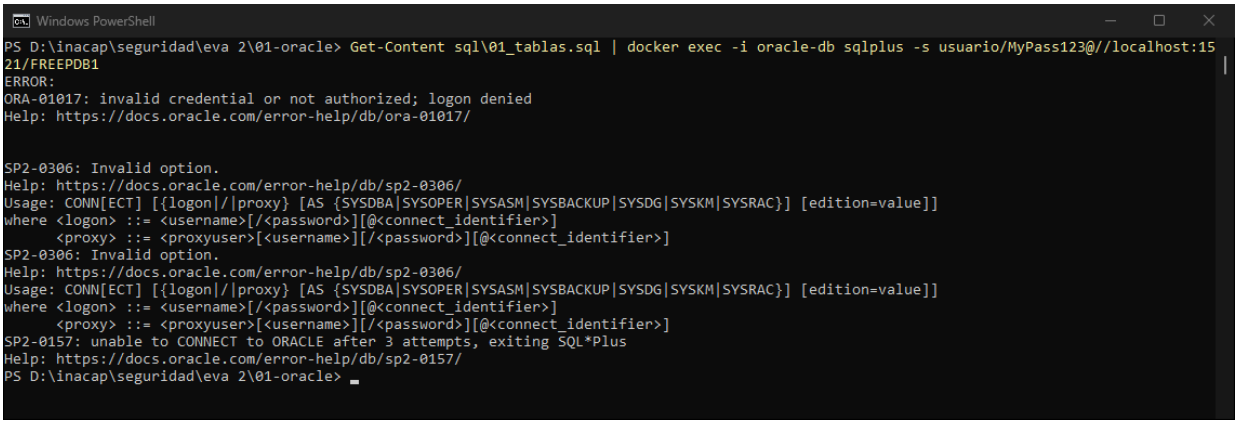


Figura 14. Primer intento de crear las tablas: error ORA-01017 (credenciales inválidas).

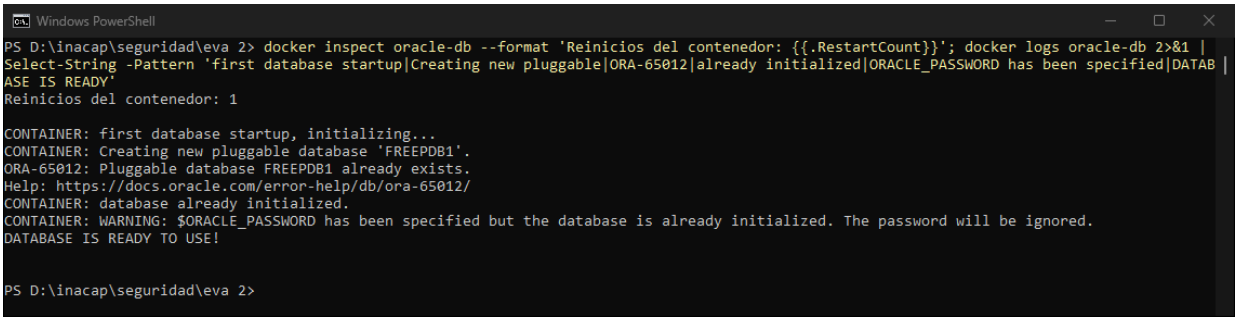


Figura 15. Causa: ORA-65012 en el primer arranque, reinicio del contenedor y omisión del usuario de aplicación.

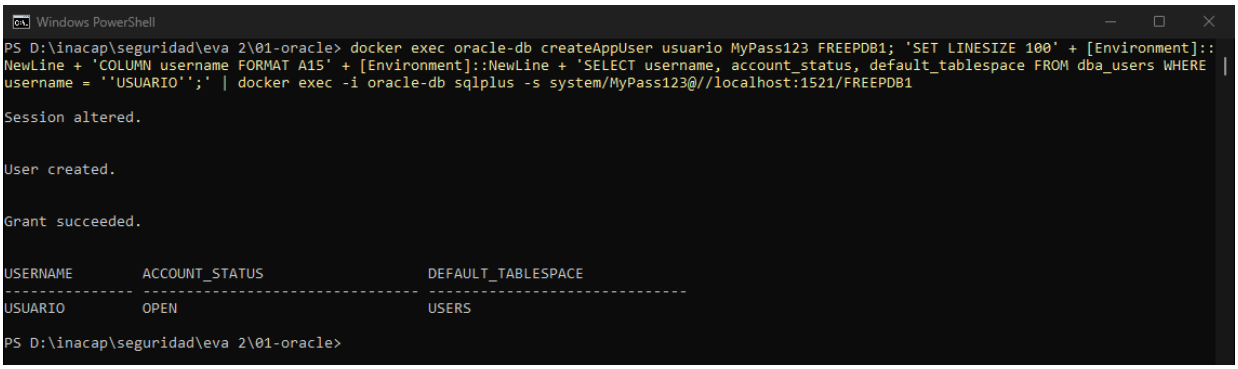


Figura 16. Creación del usuario de aplicación con createAppUser y verificación en DBA_USERS.

Con el usuario disponible se ejecutó el script `01-oracle/sql/01_tablas.sql`, que crea las tablas `CATEGORIA(id, nombre)` y `PRODUCTO(codigo, nombre, precio, stock,`

`unidad, categoriaid)` con dos registros cada una (Figura 17), y se verificó su contenido, incluida una consulta que une ambas tablas (Figura 18).

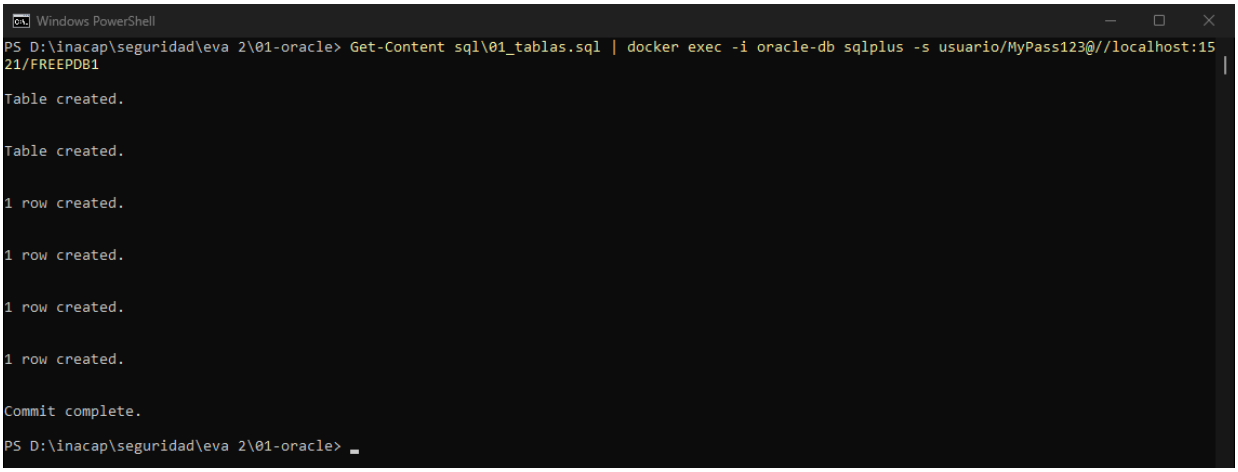


Figura 17. Creación de las tablas CATEGORIA y PRODUCTO e inserción de dos registros en cada una.

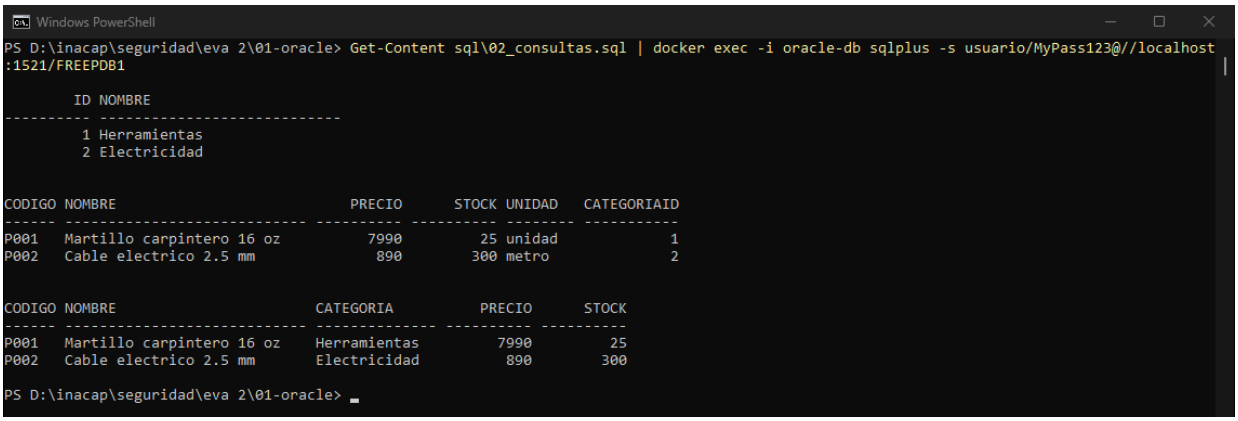


Figura 18. Consulta de las tablas y unión de producto con su categoría.

5.4 Aplicación web con Dockerfile

La aplicación es un catálogo de la ferretería desarrollado en Python 3.12 con Django 5.2 (versión de soporte extendido). Lista los productos con su categoría, permite buscarlos por nombre, muestra el detalle de cada uno, incluye el panel de administración de Django y una ruta `/salud/` que confirma la conexión con Oracle. Se conecta a la base de datos con el driver `oracledb` en modo *Thin*, que no requiere instalar Oracle Instant Client (Oracle, s.f.), usando la cadena `oracle-db:1521/FREEPDB1`: `oracle-db` es el nombre del contenedor, que el DNS de `soc-net` resuelve. La Figura 19 muestra la estructura del proyecto.

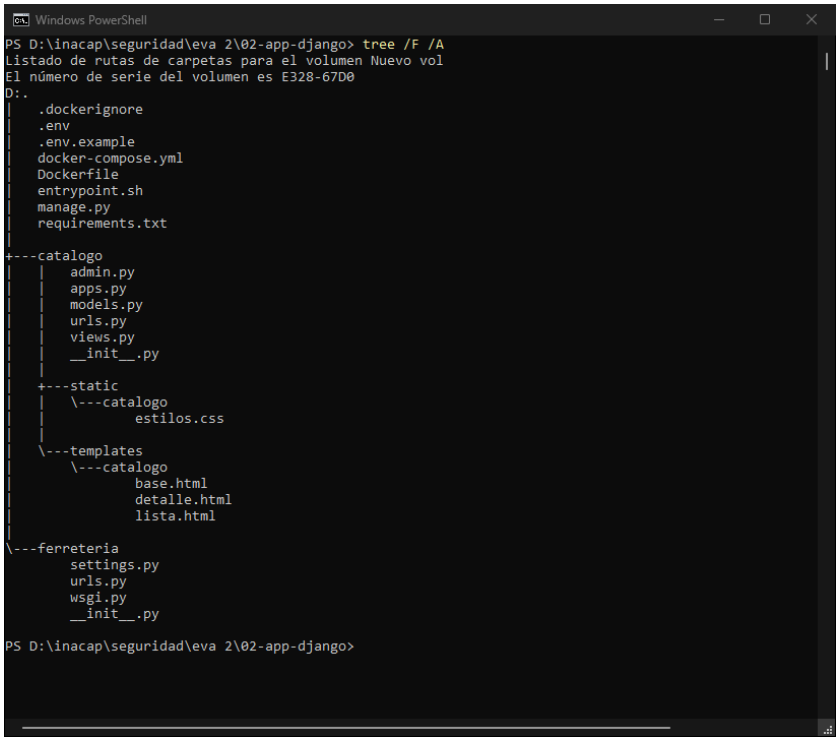


Figura 19. Estructura del proyecto Django (carpeta 02-app-django).

El Dockerfile (Figura 20) parte de la imagen oficial `python:3.12-slim`, instala las dependencias en una capa propia para aprovechar la caché, copia el código (el archivo `.dockerignore` excluye el `.env` con los secretos), reúne los archivos estáticos, crea un usuario sin privilegios para no ejecutar la aplicación como *root*, documenta el puerto 8000 y define un chequeo de salud. Su `ENTRYPOINT` ejecuta un script que espera a que Oracle acepte conexiones —necesario porque Oracle se levanta con otro archivo Compose y `depends_on` no aplica— y aplica las migraciones de Django; luego cede el control al `CMD`, que inicia el servidor gunicorn. Cada instrucción se explica en detalle en `Dockerfile.md`.

```
Windows PowerShell
PS D:\inacap\seguridad\eva 2\02-app-django> Get-Content -Encoding UTF8 Dockerfile
# Imagen base oficial de Python 3.12 en su variante "slim" (Debian reducido, más liviana).
FROM python:3.12-slim

# No generar archivos .pyc y mostrar los logs de Python al instante (sin búfer).
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# Directorio de trabajo dentro de la imagen; los comandos siguientes se ejecutan aquí.
WORKDIR /app

# Se copian primero solo las dependencias: si no cambian, Docker reutiliza esta capa en cache.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Se copia el código de la aplicación (lo listado en .dockerignore, como .env, queda fuera).
COPY . .

# Reúne los archivos estáticos (CSS, admin) en /app/staticfiles para que WhiteNoise los sirva.
# La clave usada aquí solo existe durante la construcción; la real llega por variable de entorno.
RUN DJANGO_SECRET_KEY=solo-para-construir python manage.py collectstatic --noinput

# Usuario sin privilegios: la aplicación no se ejecuta como root dentro del contenedor.
RUN useradd --create-home --uid 10001 app && chmod +x entrypoint.sh
USER app

# Documenta el puerto en que escucha gunicorn dentro del contenedor (no lo publica).
EXPOSE 8000

# Docker revisa cada 30 s que la aplicación responda en /salud/.
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/salud/', timeout=4)" || exit 1

# entrypoint.sh espera a Oracle y aplica las migraciones; luego ejecuta el CMD.
ENTRYPOINT ["./entrypoint.sh"]

# Servidor de producción gunicorn con 3 procesos, escuchando en todas las interfaces del contenedor.
CMD ["gunicorn", "ferreteria.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "3", "--access-logfile", "-"]
PS D:\inacap\seguridad\eva 2\02-app-django>
```

Figura 20. Dockerfile comentado de la aplicación web.

La imagen se construyó con `docker build -t ferreteria-web:1.0 .` en 79 segundos (Figura 21) y quedó disponible junto a las imágenes descargadas (Figura 22). Esta es la única imagen que no existe en Docker Hub.

```
Windows PowerShell
PS D:\inacap\seguridad\eva 2\02-app-django> docker build -t ferreteria-web:1.0 .
[+] Building 79.4s (12/12) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                  0.1s
 => => transferring dockerfile: 1.80kB                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                   2.4s
 => [internal] load .dockerignore                                                                                     0.1s
 => => transferring context: 151B                                                                                     0.0s
 => [internal] load build context                                                                                     0.2s
 => => transferring context: 15.87kB                                                                                  0.0s
 => [1/7] FROM docker.io/library/python:3.12-slim@sha256:a0f57064cbdcd6285fa45f51237ab0b35ea8ebaed06a1f5973e89e19dfb86704  0.4s
 => => resolve docker.io/library/python:3.12-slim@sha256:a0f57064cbdcd6285fa45f51237ab0b35ea8ebaed06a1f5973e89e19dfb86704  0.1s
 => [2/7] WORKDIR /app                                                                                                0.9s
 => [3/7] COPY requirements.txt .                                                                                     0.2s
 => [4/7] RUN pip install --no-cache-dir -r requirements.txt                                                         68.1s
 => [5/7] COPY . .                                                                                                    0.2s
 => [6/7] RUN DJANGO_SECRET_KEY=solo-para-construir python manage.py collectstatic --noinput                          1.5s
 => [7/7] RUN useradd --create-home --uid 10001 app && chmod +x entrypoint.sh                                         0.6s
 => exporting to image                                                                                                4.4s
 => => exporting layers                                                                                               2.4s
 => => exporting manifest sha256:39be17f6ff78c2d4369e92e273f4002d8e9c07cc41f15266f7613f10aa84bcaa                      0.0s
 => => exporting config sha256:df2e621bad11e003526ba917530dea3ddc79a51c3ac27a6c96041c6758996e38                       0.0s
 => => exporting attestation manifest sha256:991eea3c45281d44b6b5c0d7bbc19f5d5a1d841992d89ed487f7df82a067c5c8         0.0s
 => => exporting manifest list sha256:9f6e795a35e171dbad1b99550e6613ea7bd205dc79c9191337991d715f66dd5f                0.0s
 => => naming to docker.io/library/ferreteria-web:1.0                                                                 0.0s
 => => unpacking to docker.io/library/ferreteria-web:1.0                                                              1.7s
PS D:\inacap\seguridad\eva 2\02-app-django>
```

Figura 21. Construcción de la imagen ferreteria-web:1.0 (12 de 12 pasos).

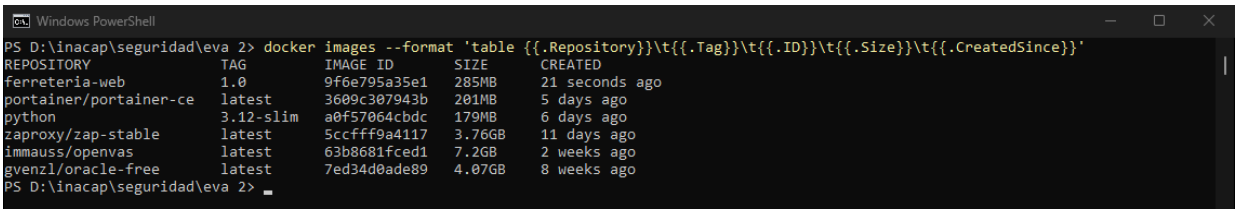


Figura 22. Imágenes disponibles: la imagen propia y las descargadas.

El archivo `02-app-django/docker-compose.yml` (Figura 23) publica el puerto 8000 del contenedor en el 8088 del equipo, carga las variables desde `.env` y asigna la IP fija `172.28.0.20` en `soc-net`, para que los escáneres tengan un objetivo estable. Al levantarla (Figura 24), los registros muestran la conexión con Oracle, las migraciones de Django, la creación del superusuario y el inicio de gunicorn (Figura 25).

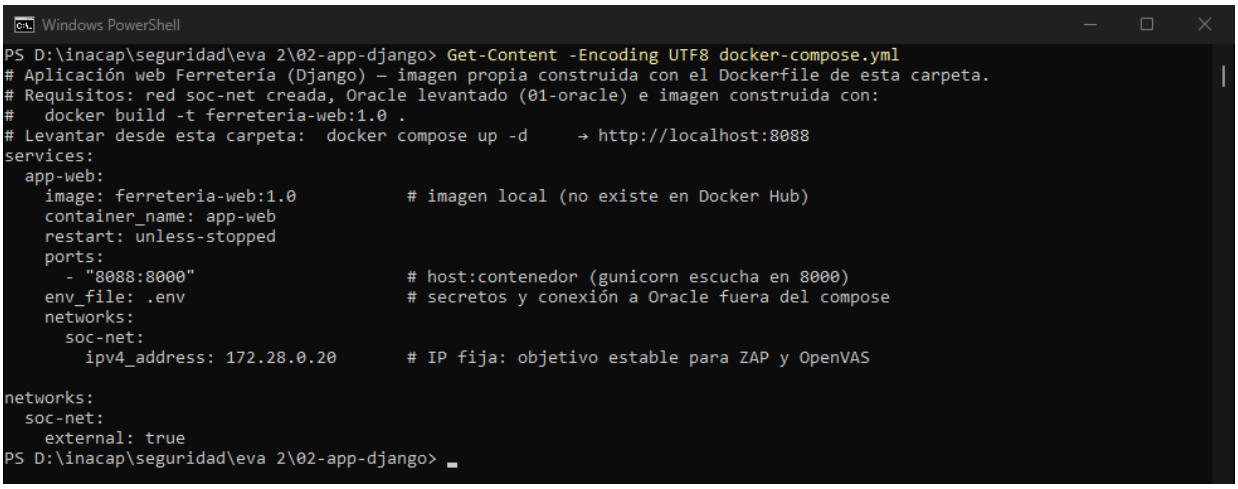


Figura 23. Archivo docker-compose.yml de la aplicación web.

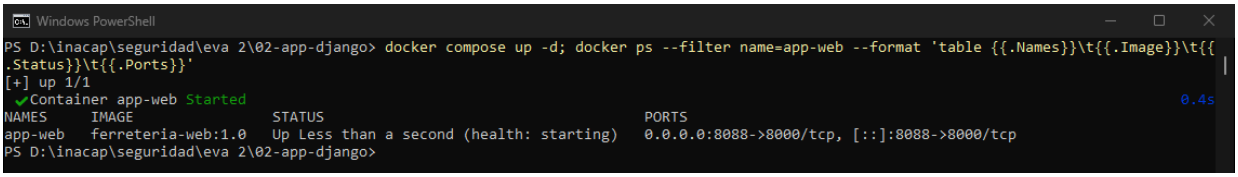


Figura 24. Levantamiento del contenedor app-web.

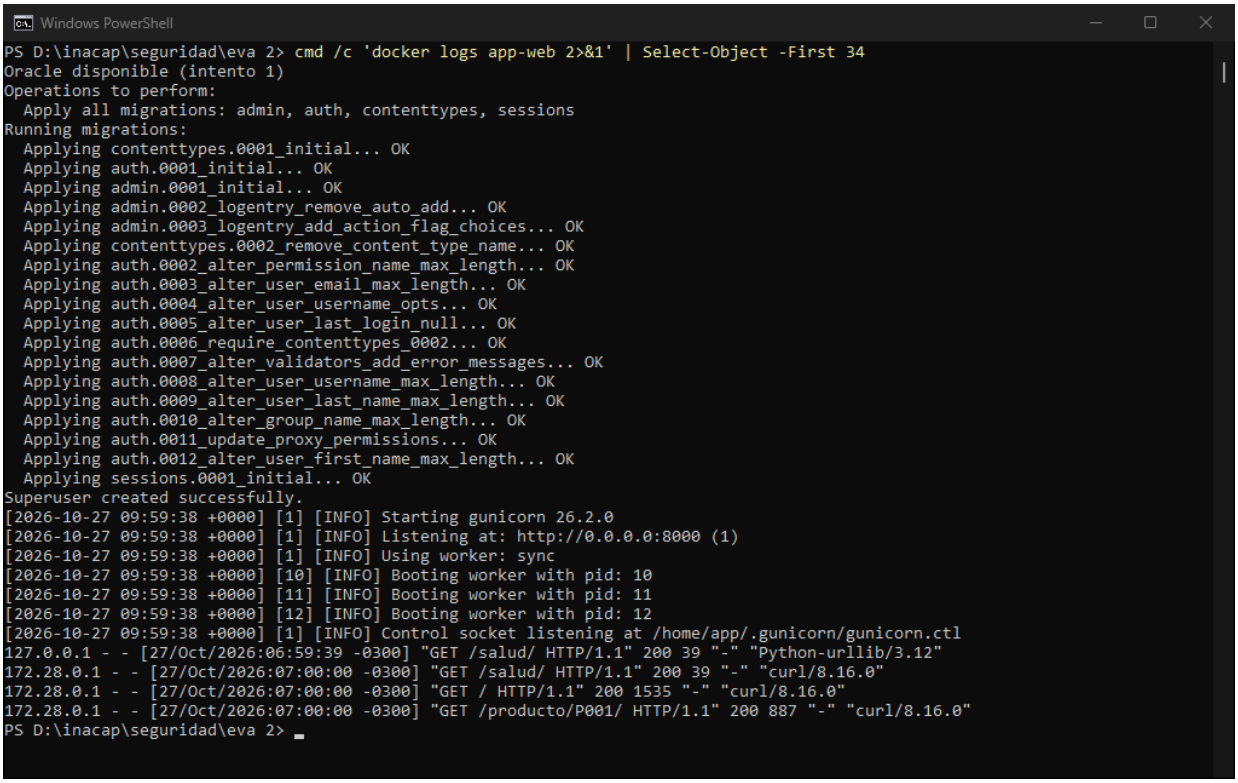


Figura 25. Registros de app-web: Oracle disponible, migraciones, superusuario y gunicorn.

Las Figuras 26 a 30 muestran la aplicación funcionando en `http://localhost:8088`: el listado leído desde Oracle, la búsqueda, el ingreso al panel de administración y la gestión de productos.



Figura 26. Catálogo de productos leído desde Oracle (http://localhost:8088).

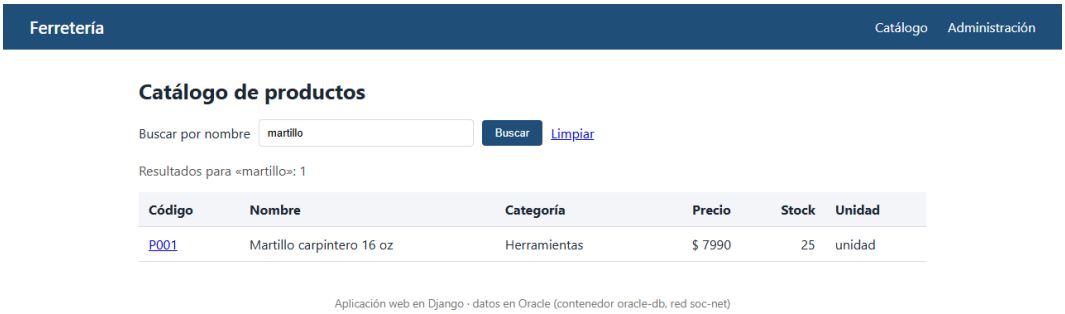


Figura 27. Búsqueda de productos por nombre.

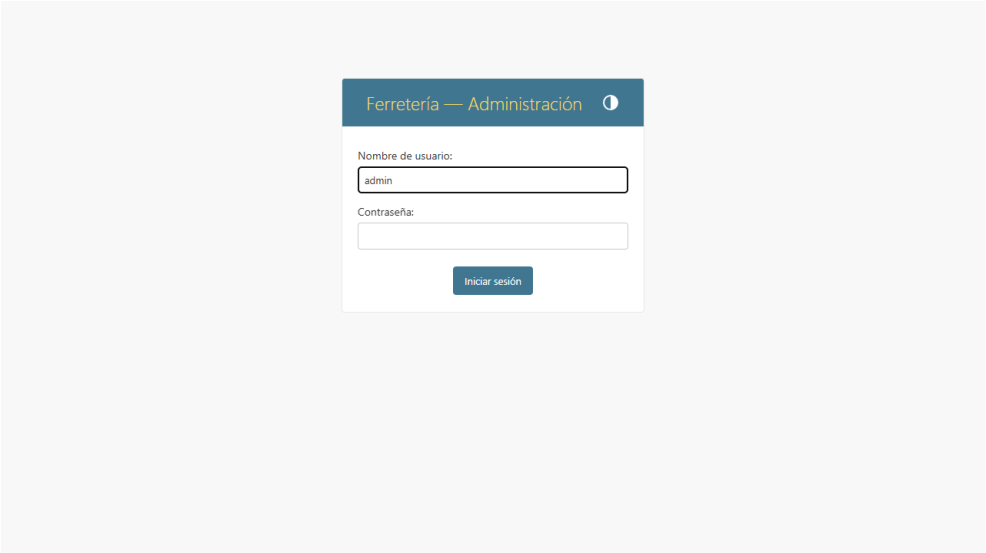


Figura 28. Formulario de ingreso al panel de administración (/admin/).



Figura 29. Panel de administración con los modelos del catálogo.

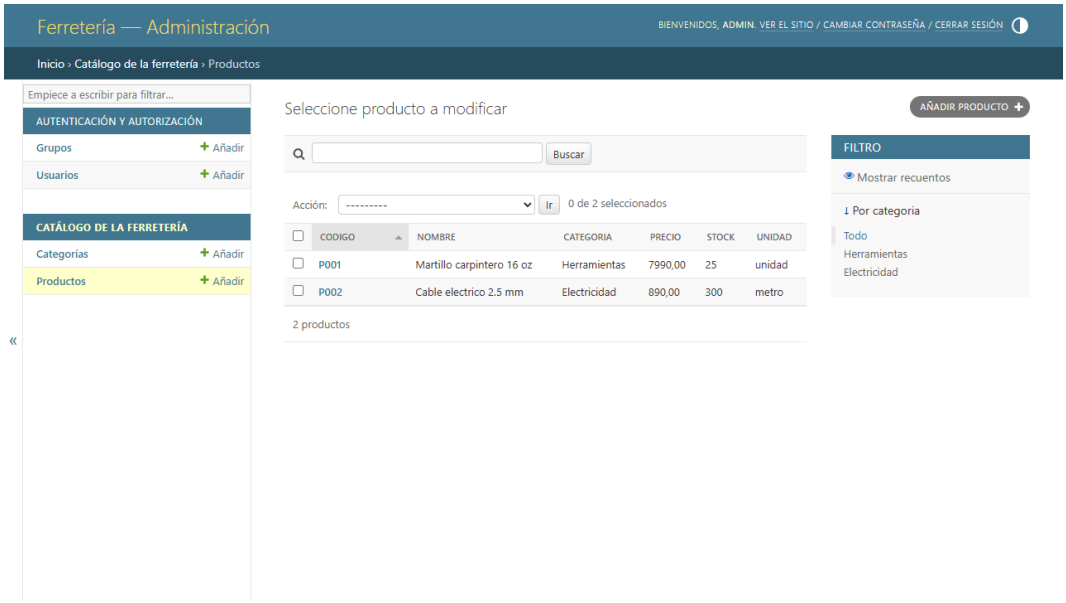


Figura 30. Productos de Oracle administrados desde el panel de Django.

5.5 Contenedor ZAP

El archivo `03-zap/docker-compose.yml` (Figura 31) usa la imagen oficial `zaproxy/zap-stable` (ZAP 2.17.0, la misma versión usada por el docente) con el comando `zap-webswing.sh`, que ofrece la interfaz gráfica completa en el navegador. Publica la interfaz en el puerto 8080 y el proxy/API en el 8090; la API queda protegida con una clave leída desde

`.env`, y los reportes se guardan en la carpeta `reportes/zap` del proyecto mediante un volumen.

La Figura 33 muestra la interfaz en `http://localhost:8080/zap/`.

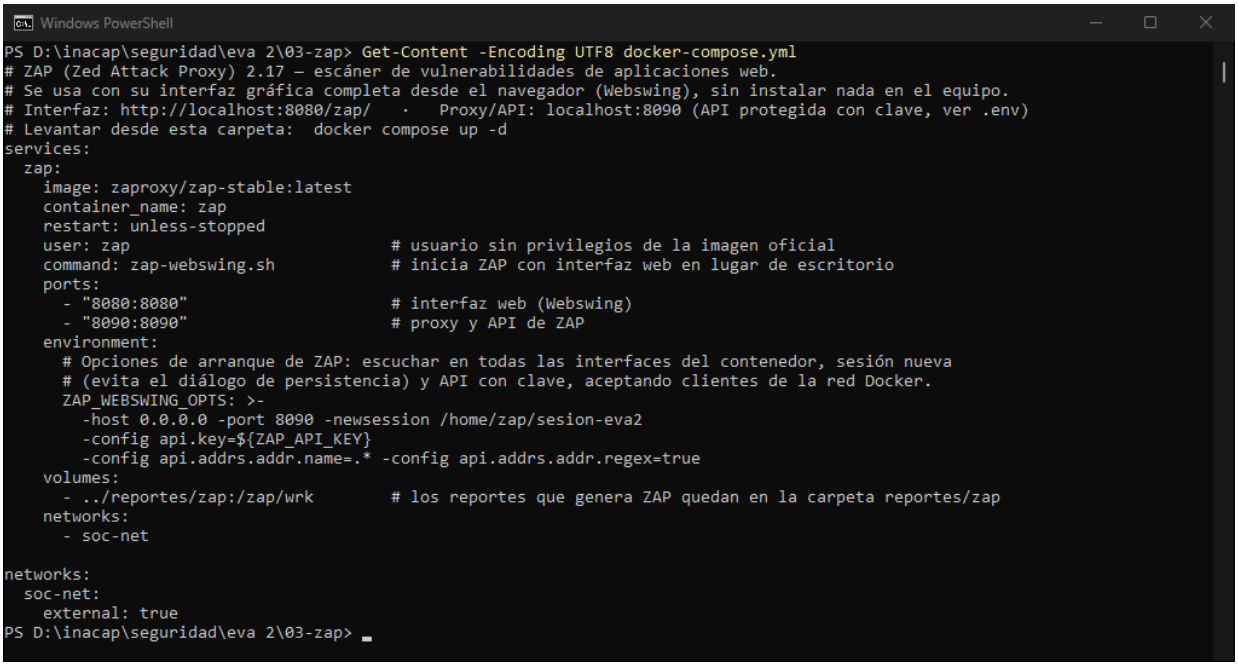


Figura 31. Archivo docker-compose.yml de ZAP.

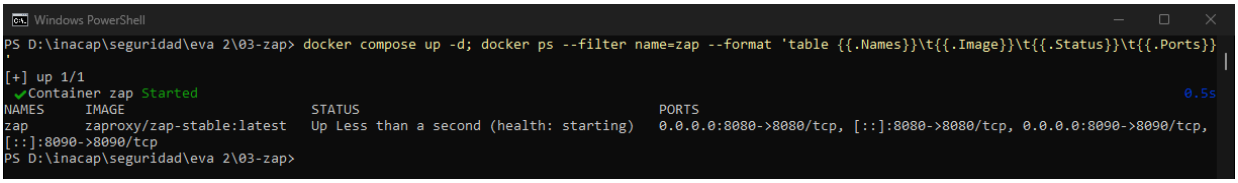


Figura 32. Levantamiento del contenedor ZAP.

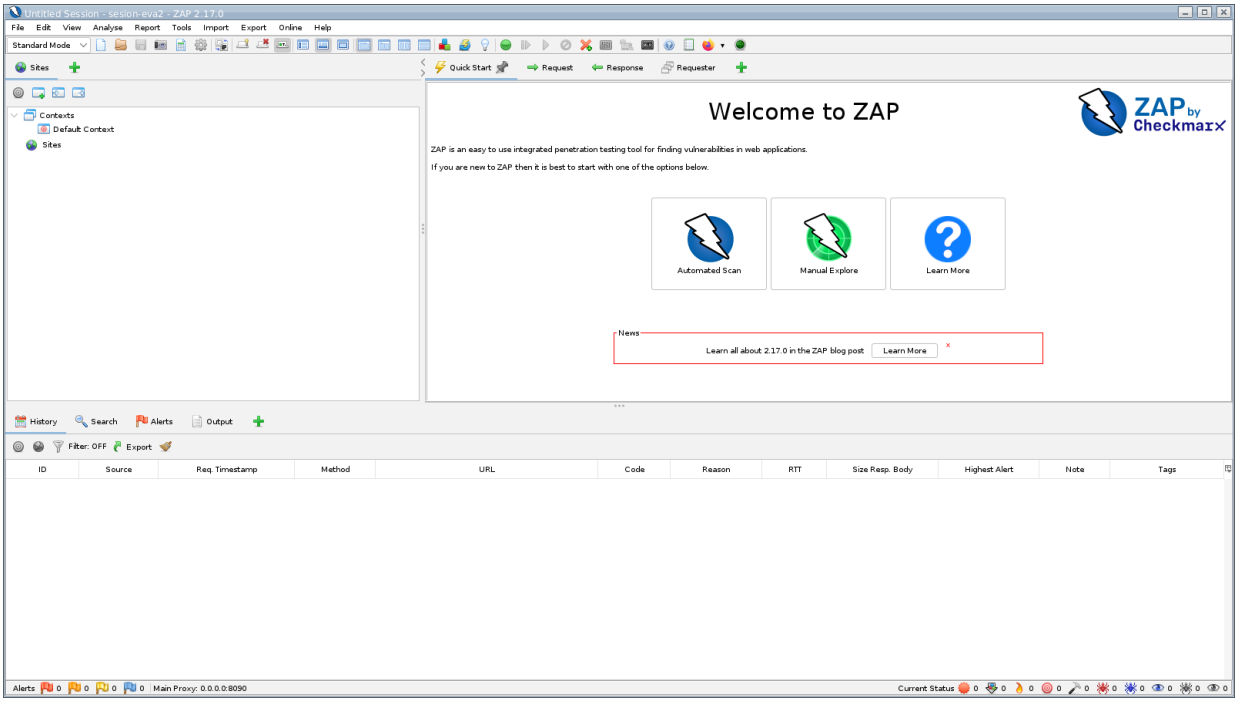


Figura 33. Interfaz de ZAP 2.17.0 en el navegador (Webswing).

5.6 Evidencia de la red: contenedores e IP

Con los cuatro contenedores en ejecución (Figura 34) se ejecutó el comando entregado por el docente para ver los contenedores de `soc-net` con su IP (Figura 35). En PowerShell de Windows las comillas internas `"\n"` deben escaparse como `\"\n\"` (o reemplazarse por `{{println}}`); el resultado es el mismo. Estas IP son las que se entregan a los escáneres. La Figura 36 muestra los volúmenes creados.

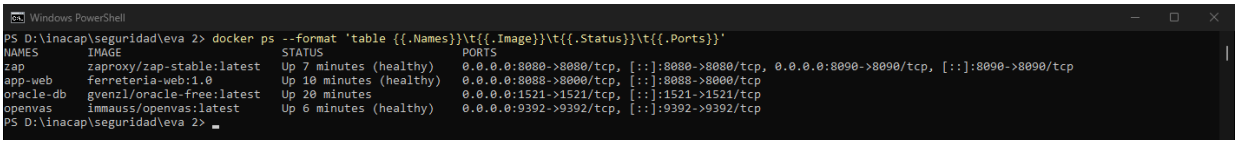


Figura 34. Los cuatro contenedores en ejecución (docker ps).

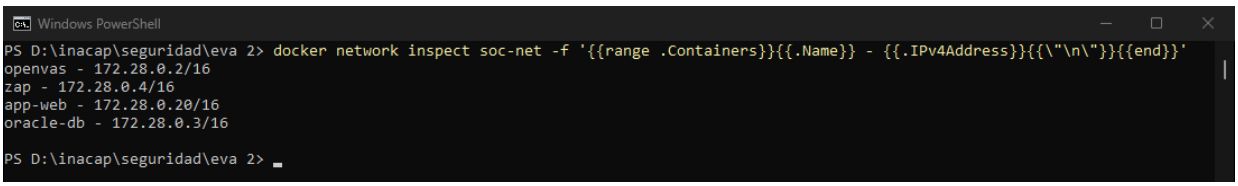


Figura 35. Contenedores de soc-net con su dirección IP (comando del docente).

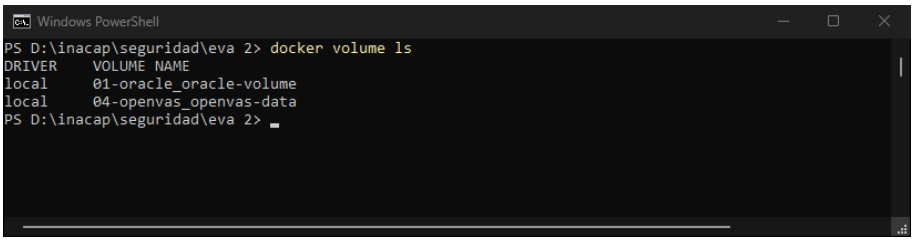


Figura 36. Volúmenes persistentes de Oracle y OpenVAS.

5.7 Diagramas de los archivos Compose

Cada archivo Compose se cargó en el visualizador de YAMLTools, reemplazando antes las contraseñas por `"[oculto]"` para no enviarlas a un sitio externo. Las Figuras 37 a 40 muestran su estructura en la vista *Diagrama de Bloques*: servicios, volúmenes y la red externa `soc-net`.

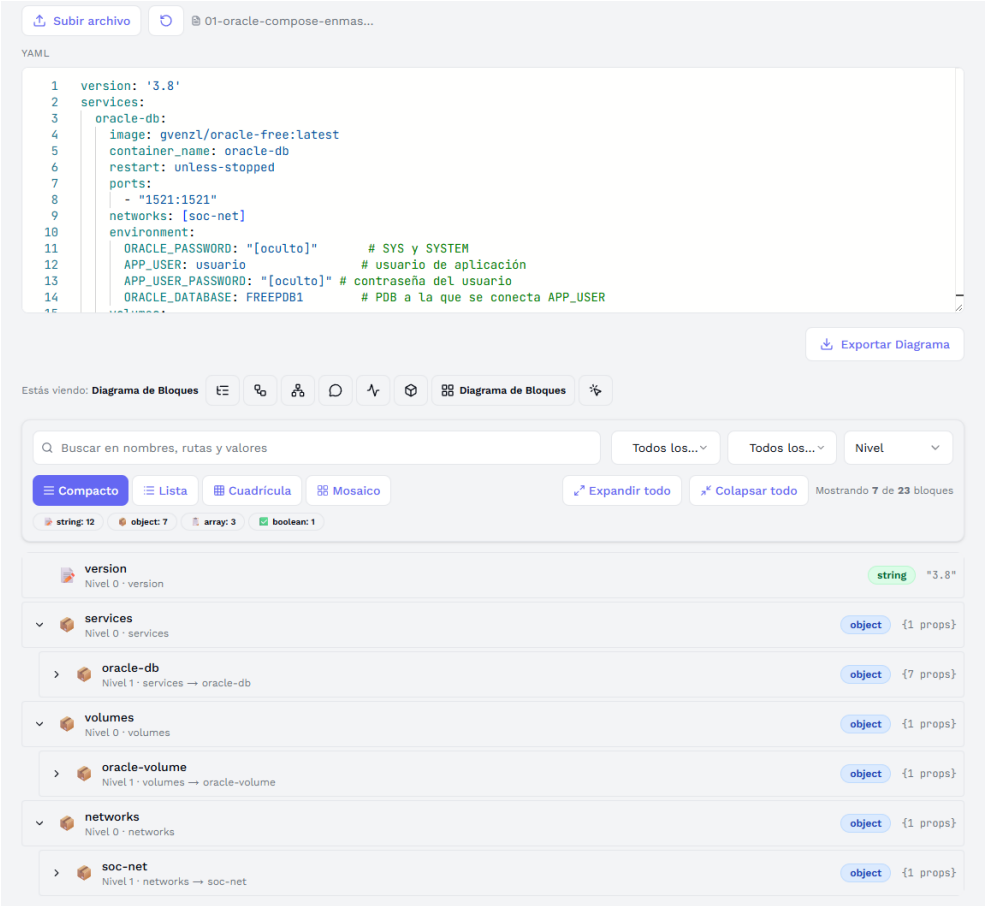


Figura 37. Diagrama de bloques del compose de Oracle (YAMLTools).

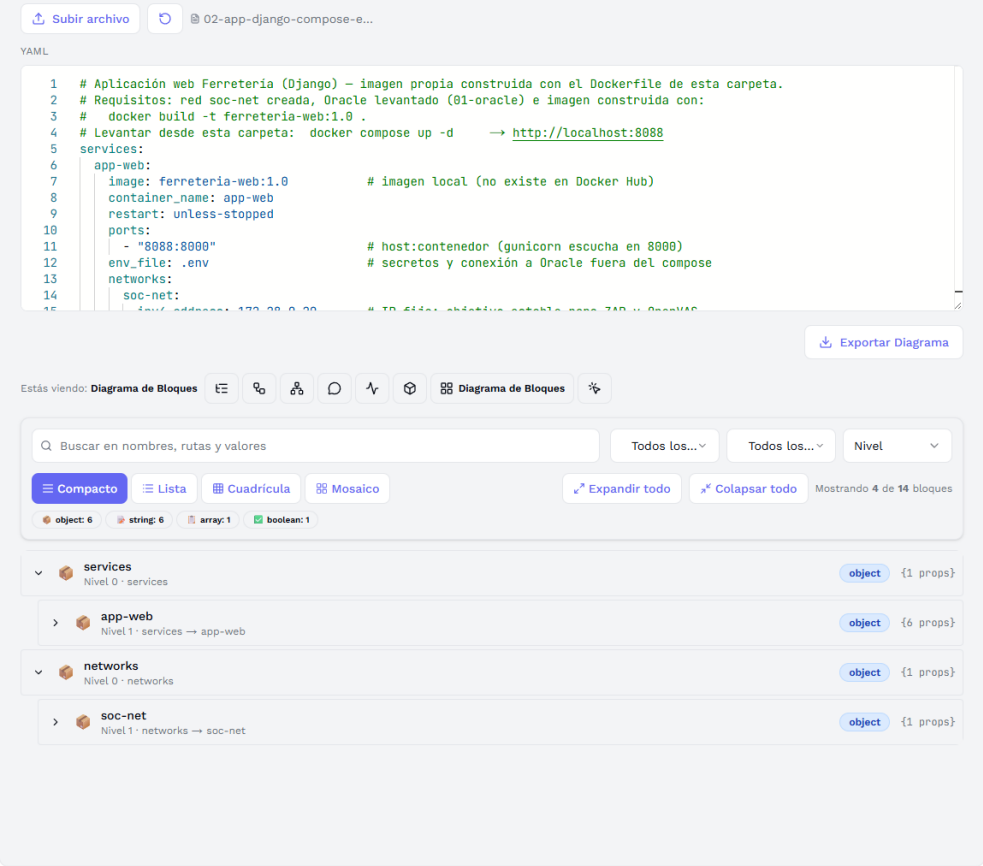


Figura 38. Diagrama de bloques del compose de la aplicación web (YAMLTools).

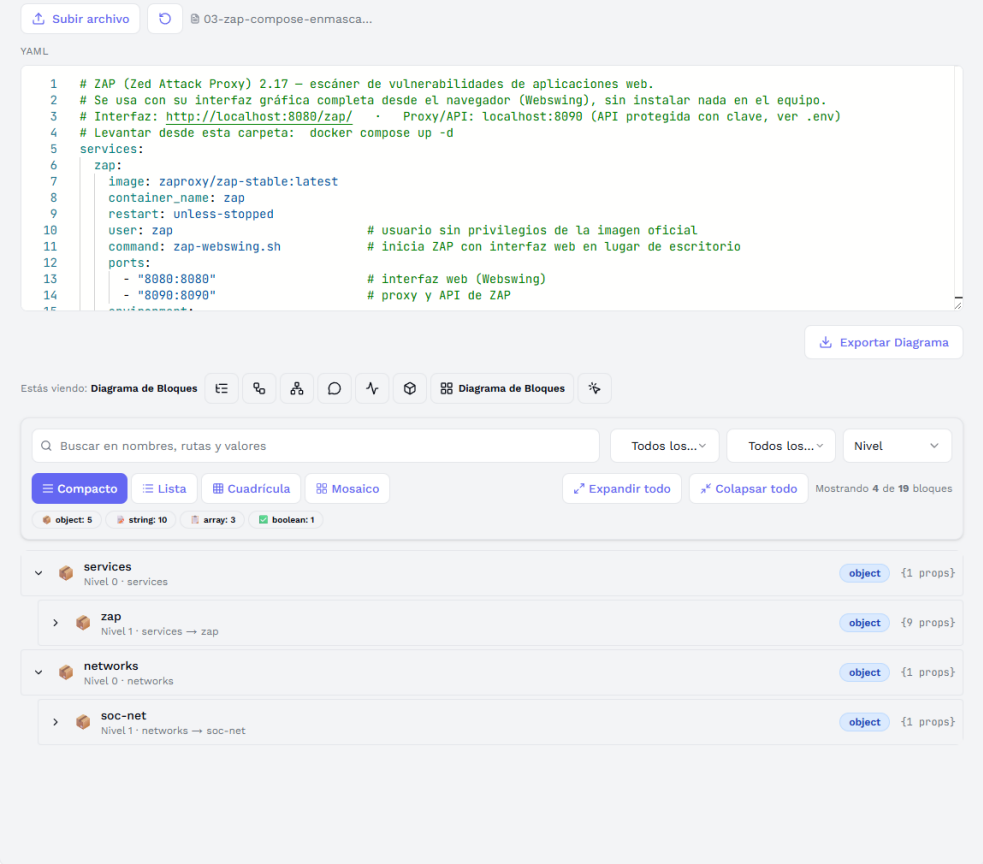


Figura 39. Diagrama de bloques del compose de ZAP (YAMLTools).

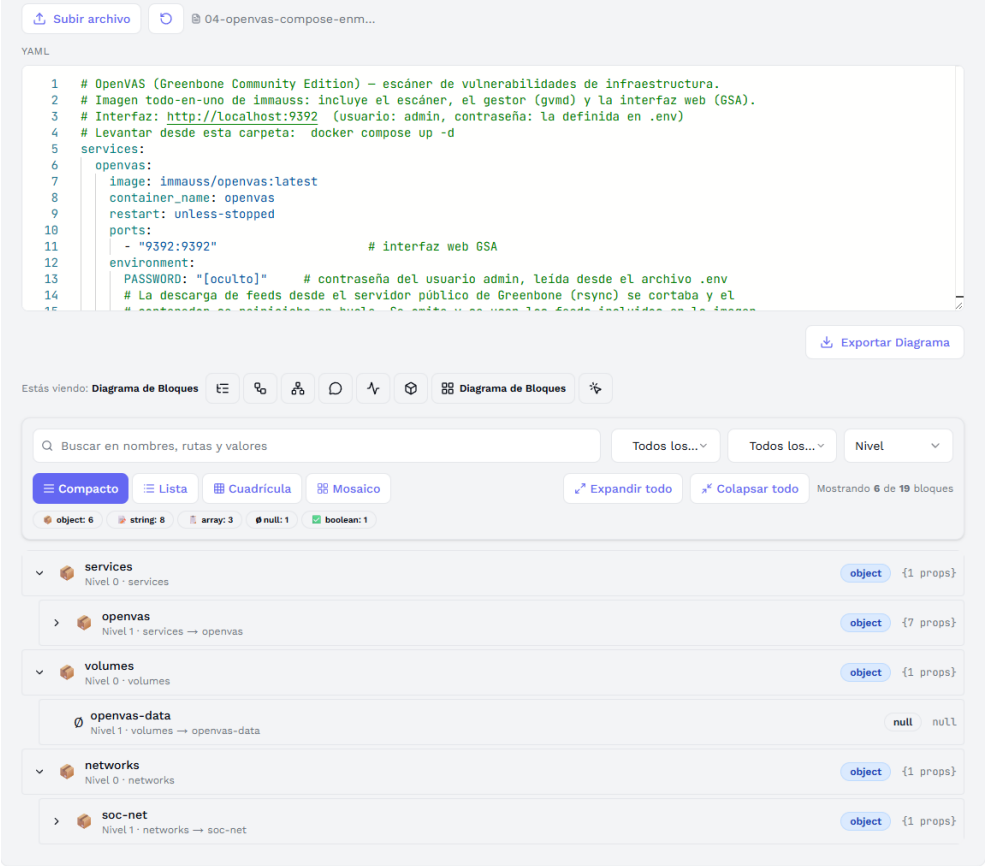


Figura 40. Diagrama de bloques del compose de OpenVAS (YAMLTools).

6. Identificación: escaneos de vulnerabilidades

Los escaneos se lanzaron desde los contenedores de seguridad hacia `http://172.28.0.20:8000`, es decir, la IP de la aplicación dentro de `soc-net` y su **puerto interno** (8000), no el 8088 publicado en el equipo. ZAP se operó mediante su API y su interfaz web muestra el avance en vivo; OpenVAS se configuró por su protocolo de gestión (GMP) y se revisó en su interfaz GSA.

6.1 ZAP — Escaneo 1: exploración y análisis pasivo

El primer escaneo ejecutó el *spider* de ZAP, que recorrió el sitio y encontró 16 URL en 11 segundos (Figura 41), y el análisis pasivo de todas las respuestas. Se obtuvieron dos alertas de riesgo Medio, una Baja y tres Informativas (Figura 42). El resultado se exportó como `Reporte1.json` y `Reporte1.html` (Figura 43).

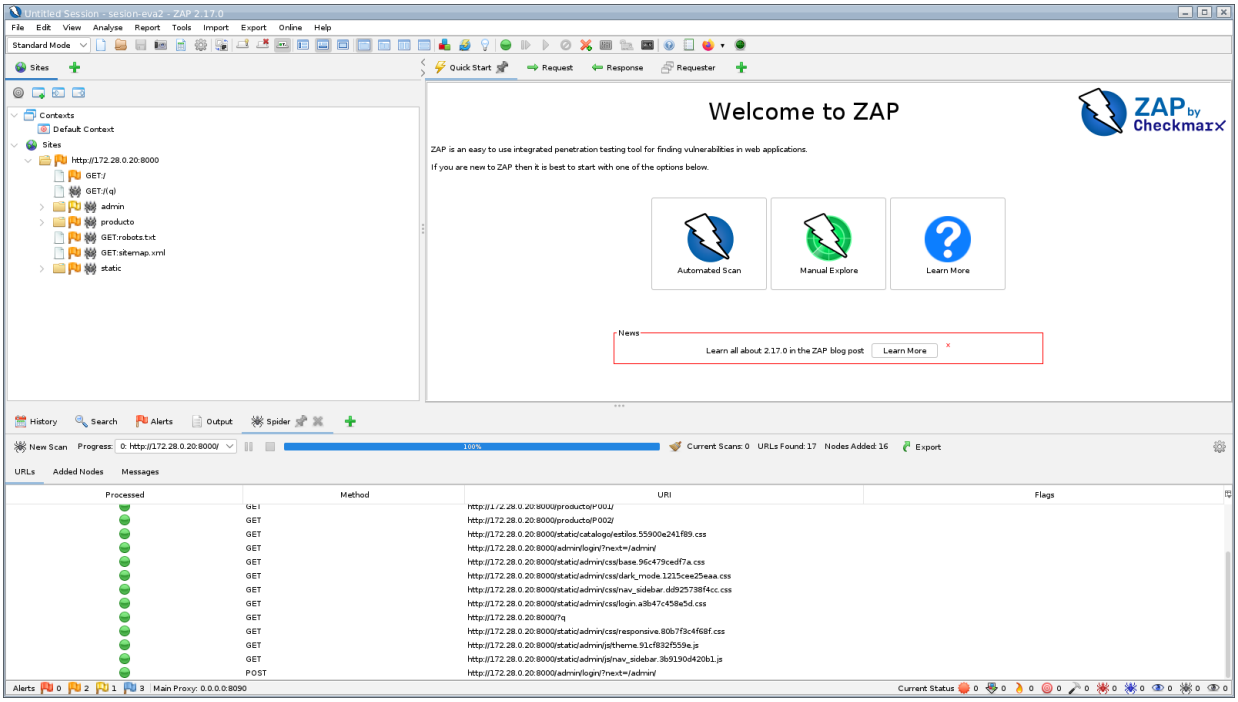


Figura 41. Escaneo 1: árbol del sitio descubierto por el spider de ZAP.

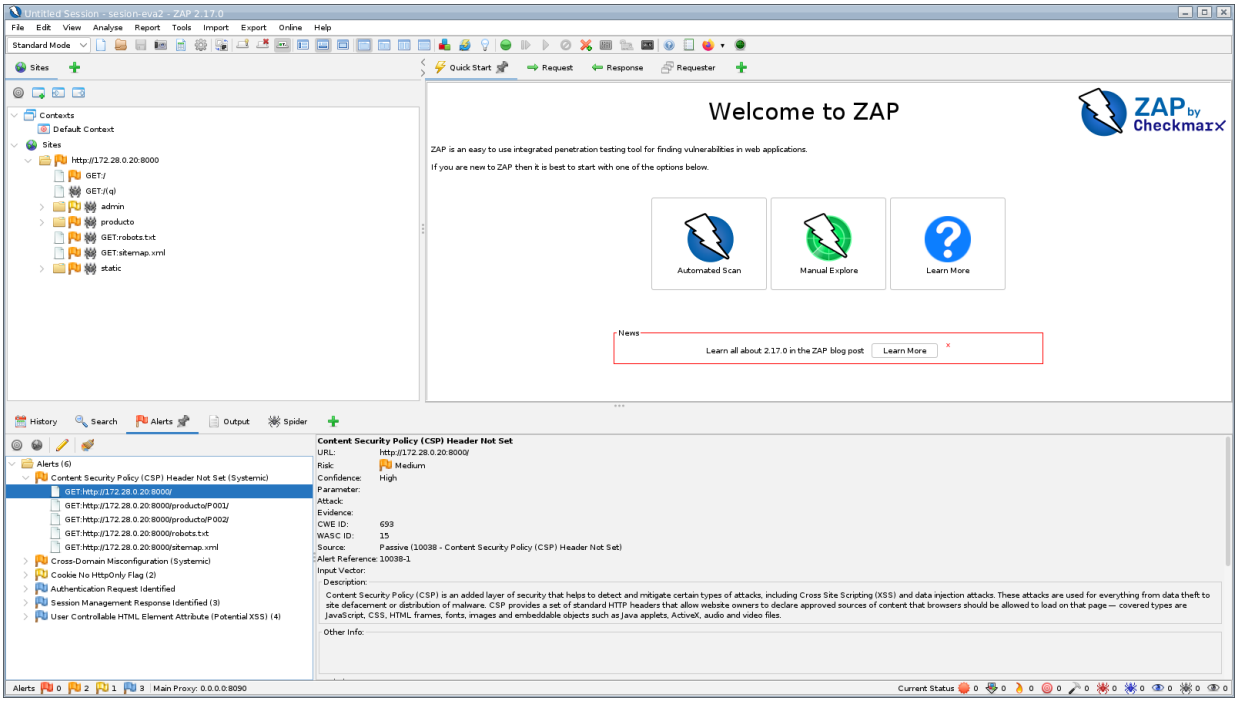


Figura 42. Escaneo 1: alertas de ZAP y detalle de «Content Security Policy (CSP) Header Not Set».

ZAP by Checkmarx **Reporte 1 - Escaneo pasivo (spider) - Ferreteria**

Site: http://172.28.0.20:8000

Generated on Tue, 27 Oct 2026 10:16:51

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	1
Informational	3
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://172.28.0.20:8000	Percentage of responses with status code 2xx	83 %
Info	Informational	http://172.28.0.20:8000	Percentage of responses with status code 3xx	5 %
Info	Informational	http://172.28.0.20:8000	Percentage of responses with status code 4xx	11 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with content type text/css	35 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with content type text/html	52 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with content type text/javascript	11 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with method GET	94 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with method POST	5 %
Info	Informational	http://172.28.0.20:8000	Count of total endpoints	17
Info	Informational	http://172.28.0.20:8000	Percentage of slow responses	5 %

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Cross-Domain Misconfiguration	Medium	Systemic
Cookie No HttpOnly Flag	Low	2
Authentication Request Identified	Informational	1
Session Management Response Identified	Informational	3

Figura 43. Reporte 1 de ZAP en HTML: resumen de alertas por nivel de riesgo.

6.2 ZAP — Escaneo 2: escaneo activo

El segundo escaneo fue activo: ZAP envió 3 444 peticiones con pruebas de ataque durante cerca de cinco minutos y medio (Figuras 44 y 45). Se intentaron, entre otras, inyecciones SQL y XSS sobre el parámetro de búsqueda `q` y el formulario de ingreso; ninguna se confirmó, y los intentos de envío al formulario sin token fueron rechazados por la protección CSRF de Django (código 403). Se agregaron las alertas «HTTP Only Site» (Medio) y «User Agent Fuzzer» (Informativa) (Figura 46). El resultado se exportó como `Reporte2.json` y `Reporte2.html` (Figuras 47 y 48).

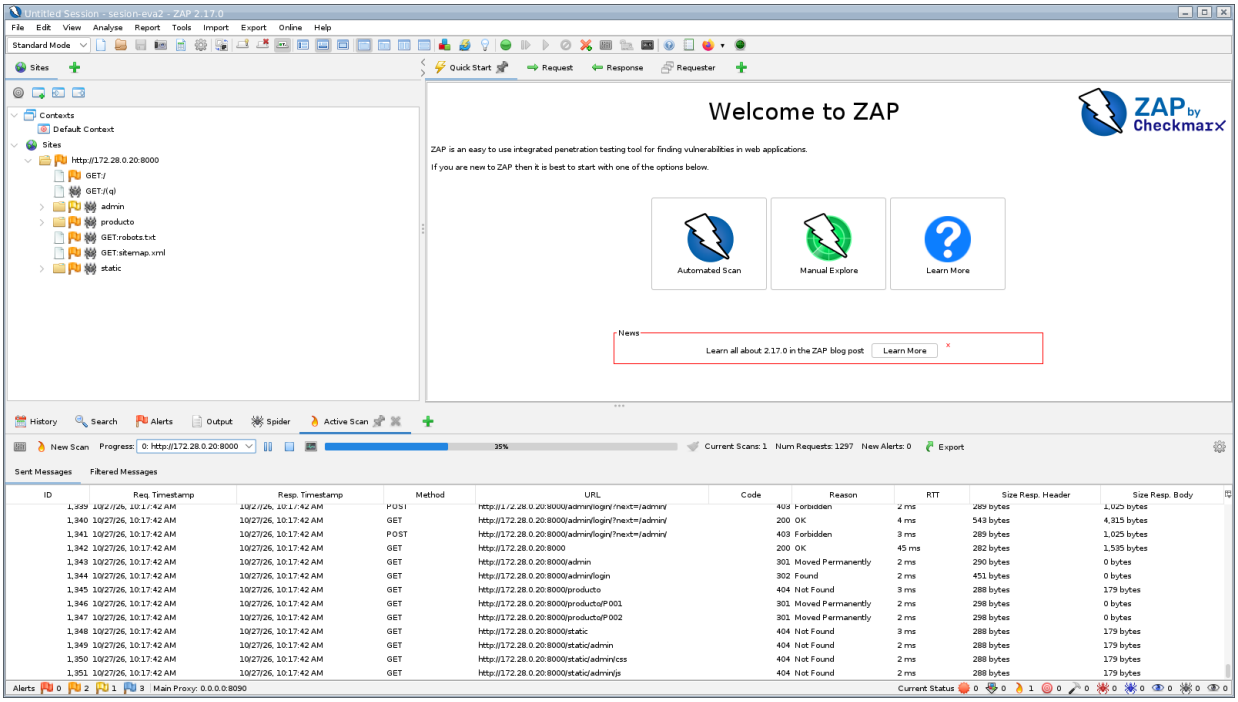


Figura 44. Escaneo 2: escaneo activo en curso (35 %).

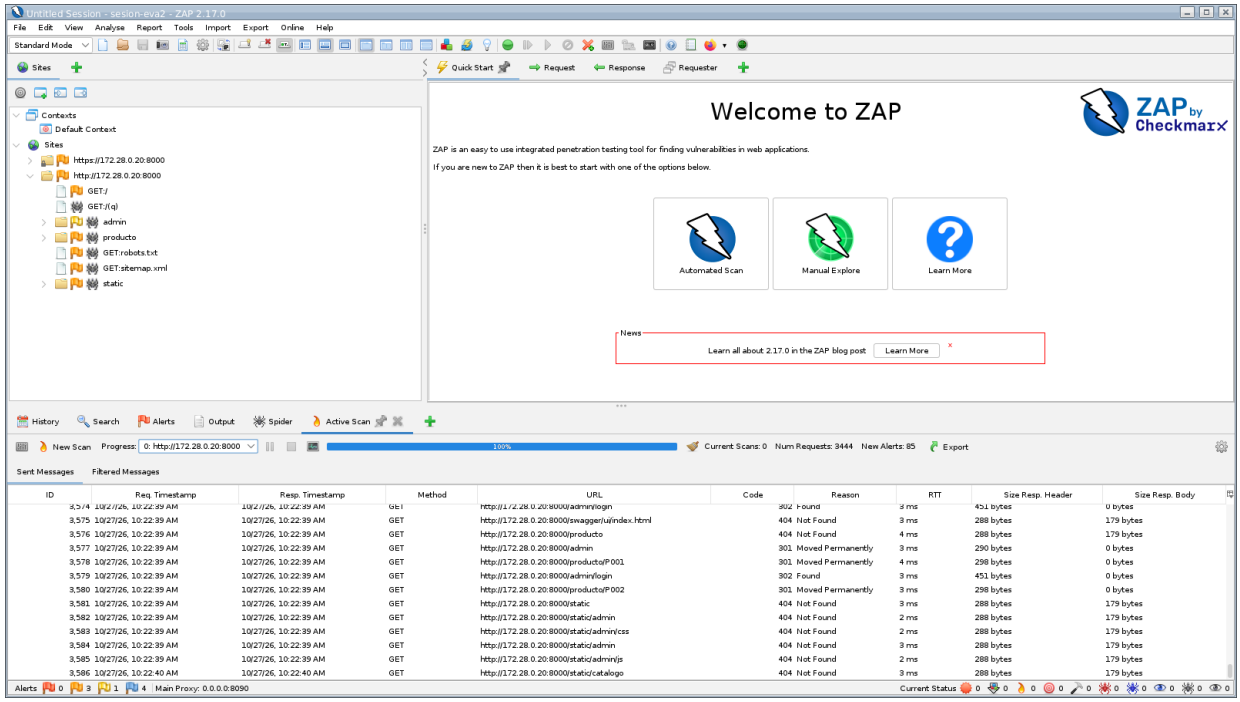


Figura 45. Escaneo 2: escaneo activo terminado (3 444 peticiones).

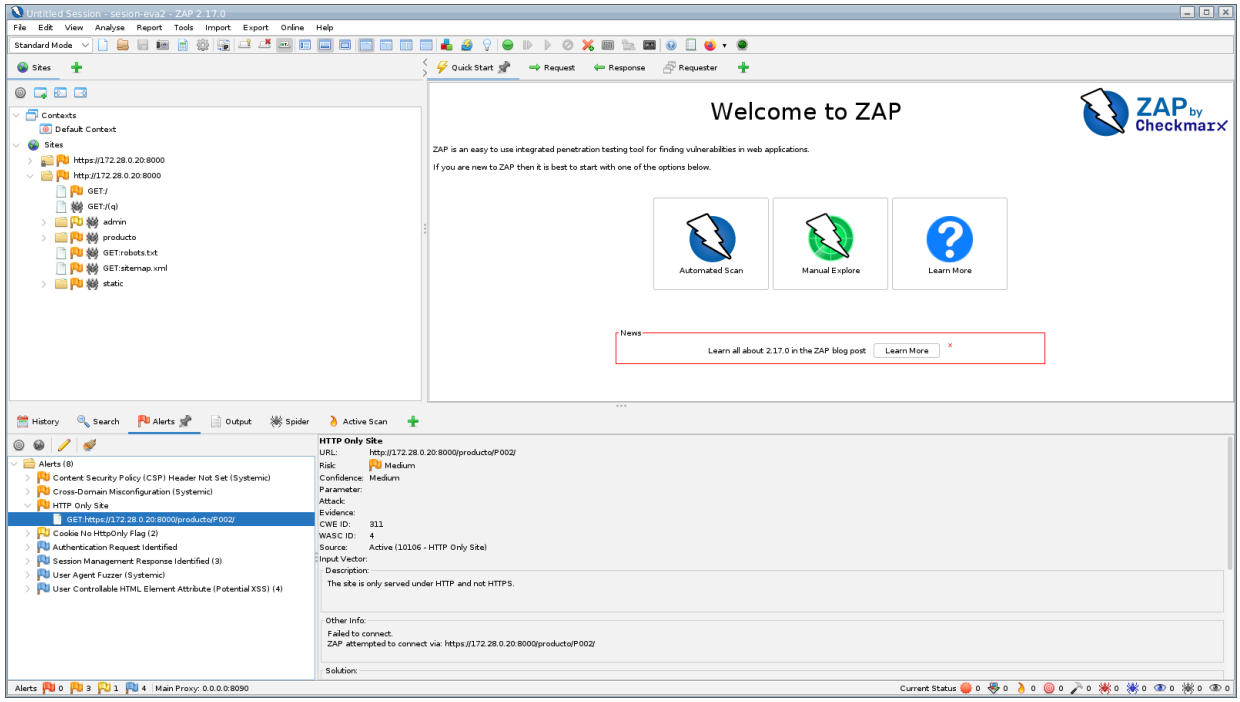
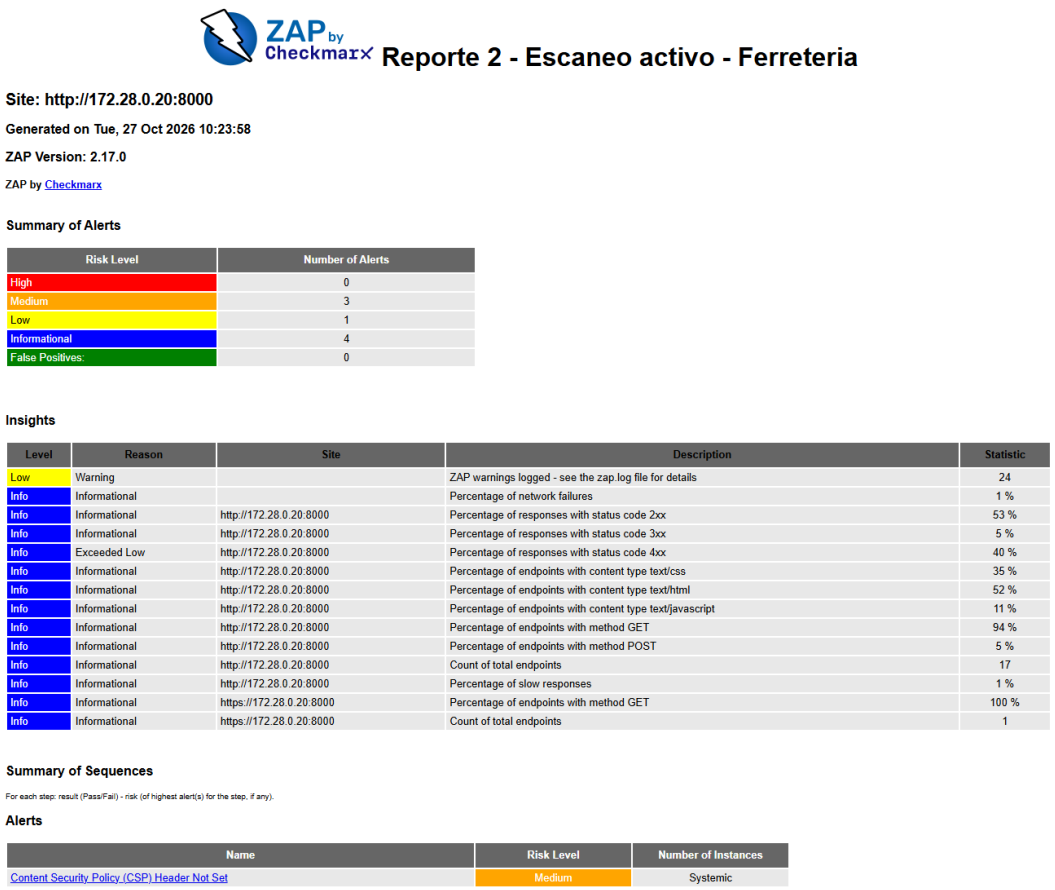


Figura 46. Escaneo 2: alerta «HTTP Only Site» detectada por el escaneo activo.



ZAP by Checkmarx Reporte 2 - Escaneo activo - Ferreteria

Site: http://172.28.0.20:8000

Generated on Tue, 27 Oct 2026 10:23:58

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	3
Low	1
Informational	4
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	24
Info	Informational		Percentage of network failures	1 %
Info	Informational	http://172.28.0.20:8000	Percentage of responses with status code 2xx	53 %
Info	Informational	http://172.28.0.20:8000	Percentage of responses with status code 3xx	5 %
Info	Exceeded Low	http://172.28.0.20:8000	Percentage of responses with status code 4xx	40 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with content type text/css	35 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with content type text/html	52 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with content type text/javascript	11 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with method GET	94 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with method POST	5 %
Info	Informational	http://172.28.0.20:8000	Count of total endpoints	17
Info	Informational	http://172.28.0.20:8000	Percentage of slow responses	1 %
Info	Informational	https://172.28.0.20:8000	Percentage of endpoints with method GET	100 %
Info	Informational	https://172.28.0.20:8000	Count of total endpoints	1

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	Systemic

Figura 47. Reporte 2 de ZAP en HTML.

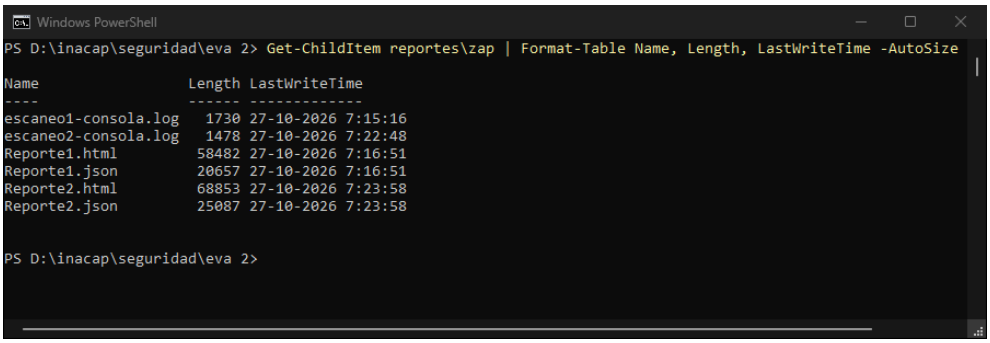


Figura 48. Reportes de ZAP generados en formato JSON y HTML.

Tabla 5

Resumen de alertas de ZAP por escaneo

Alerta (regla ZAP)	CWE	Escaneo 1	Escaneo 2	Riesgo
Content Security Policy (CSP) Header Not Set (10038)	693	Sí	Sí	**Medio**
Cross-Domain Misconfiguration (10098)	264	Sí	Sí	**Medio**
HTTP Only Site (10106)	311	—	Sí	**Medio**
Cookie No HttpOnly Flag (10010)	1004	Sí	Sí	**Bajo**

Alerta (regla ZAP)	CWE	Escaneo 1	Escaneo 2	Riesgo
Authentication Request Identified (10111)	—	Sí	Sí	Informativo
Session Management Response Identified (10112)	—	Sí	Sí	Informativo
User Controllable HTML Element Attribute (10031)	20	Sí	Sí	Informativo
User Agent Fuzzer (10104)	—	—	Sí	Informativo

Nota. Riesgo según la clasificación de ZAP. Fuente: Reporte1.json y Reporte2.json.

6.3 OpenVAS — Pasada 1

Se ingresó a la interfaz GSA en `http://localhost:9392` (Figura 49); el panel muestra las más de 194 000 NVT disponibles (Figura 50). Se creó el objetivo `app-web Ferreteria (172.28.0.20)` con la lista de puertos *All IANA assigned TCP* y la prueba de disponibilidad *Consider Alive* (Figura 51), y una tarea con la configuración *Full and fast* (Figura 52). La primera pasada duró cinco minutos y encontró dos hallazgos de severidad Baja y once registros informativos (Figuras 53 y 54). Se exportó en XML, CSV y PDF.

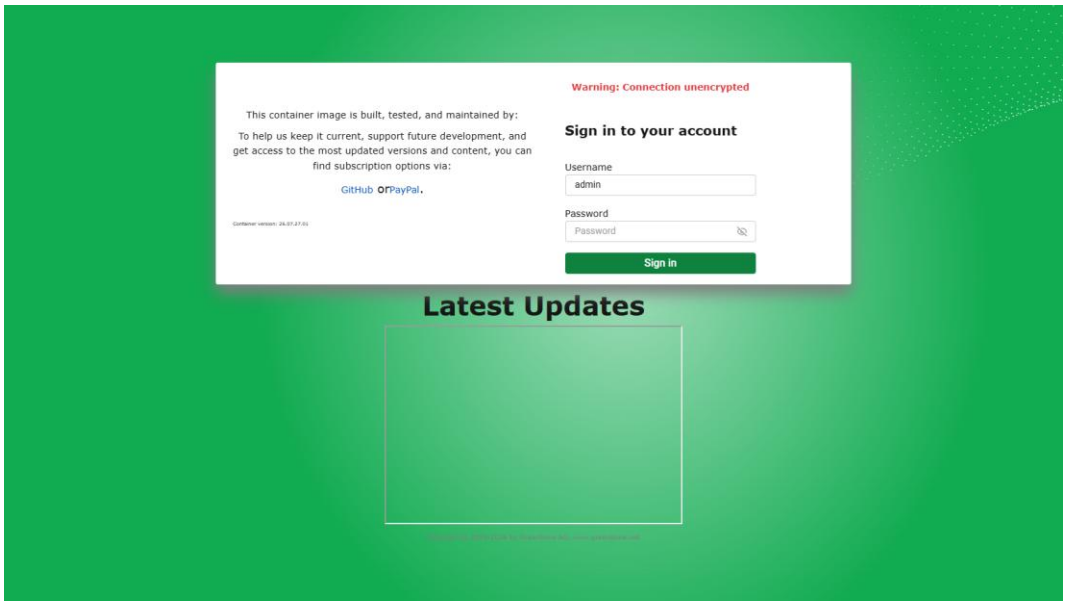


Figura 49. Ingreso a la interfaz web de OpenVAS (GSA). Se observa la advertencia de conexión sin cifrar.

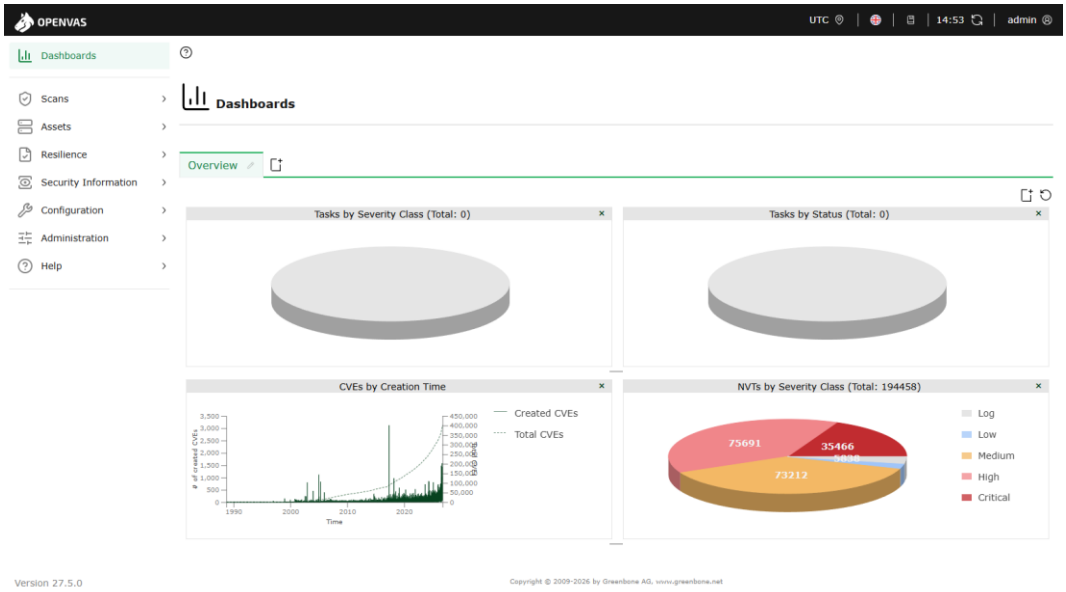


Figura 50. Panel de OpenVAS con las NVT cargadas por severidad.

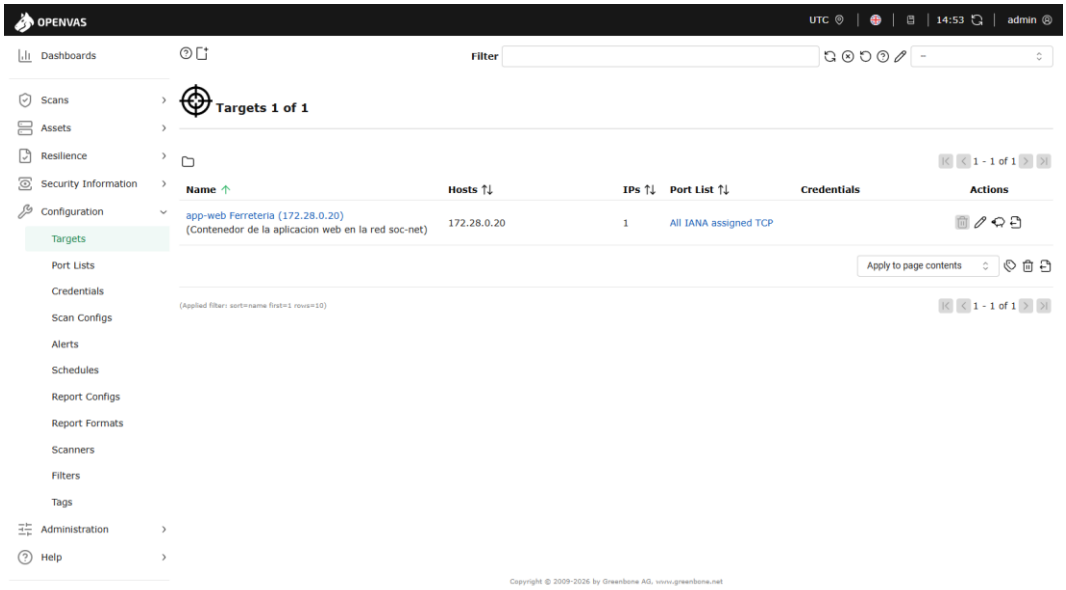


Figura 51. Objetivo creado: contenedor de la aplicación web (172.28.0.20).

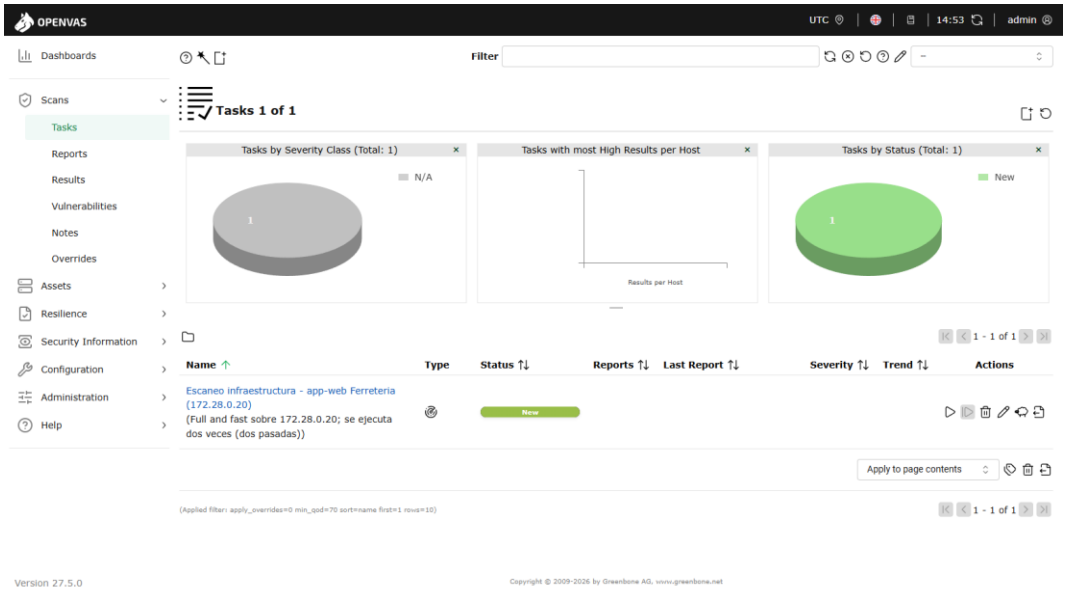


Figura 52. Tarea «Full and fast» creada sobre el objetivo.

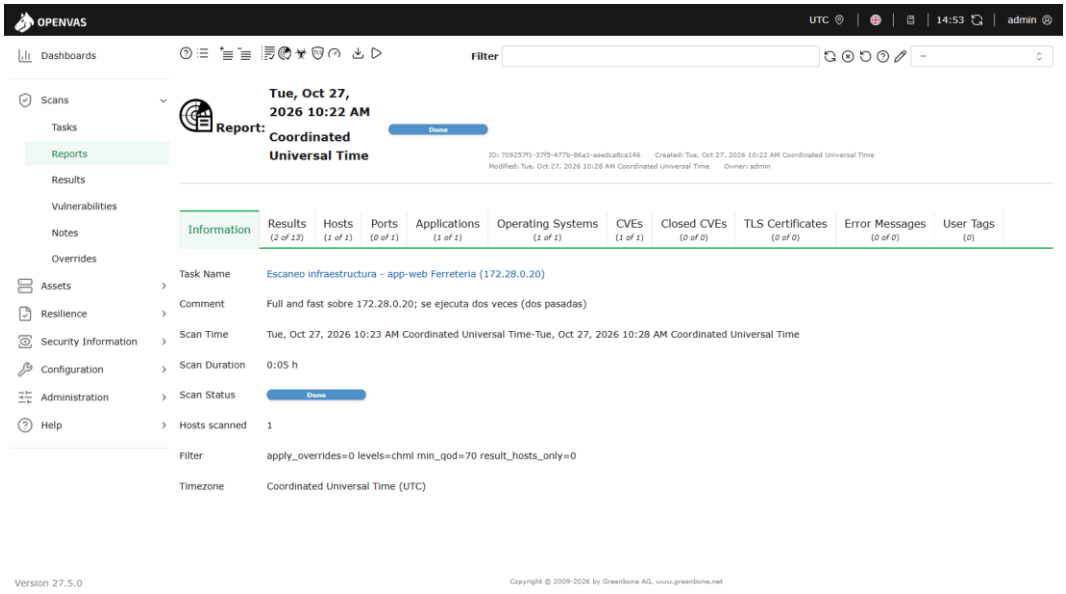


Figura 53. Pasada 1 terminada: duración, estado y cantidad de resultados.

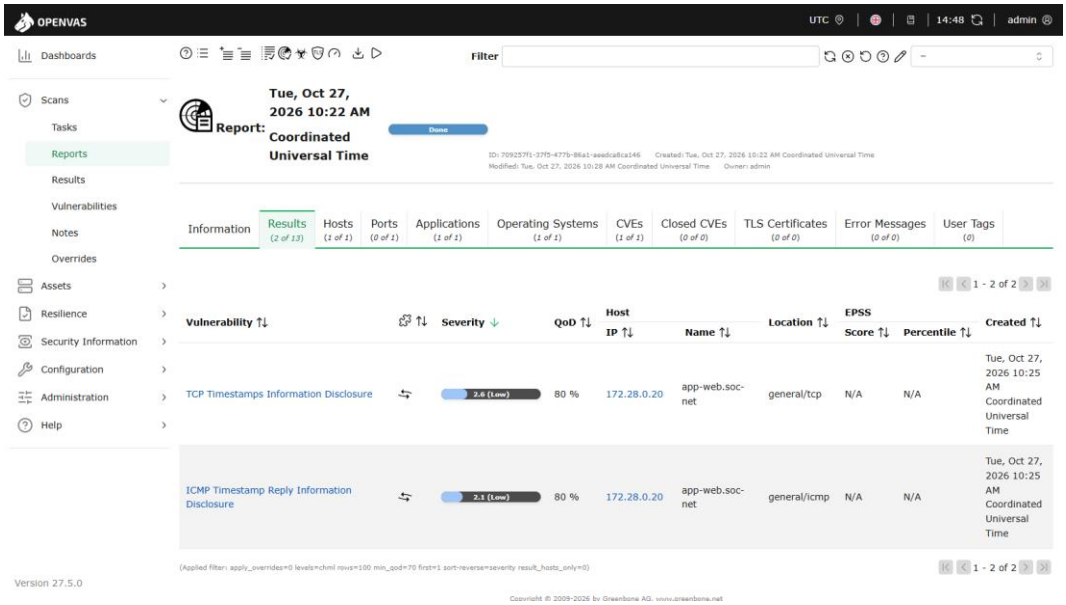


Figura 54. Pasada 1: hallazgos de severidad Baja.

6.4 OpenVAS — Pasada 2

El docente pidió dos pasadas porque una sola no siempre es confiable. La segunda pasada, sobre la misma tarea, duró seis minutos y obtuvo exactamente los mismos resultados (Figuras 55 a 57), lo que confirma que los hallazgos son consistentes; la tendencia de la tarea aparece estable («→»). La Figura 58 muestra los reportes exportados y la Tabla 6 compara ambas pasadas.

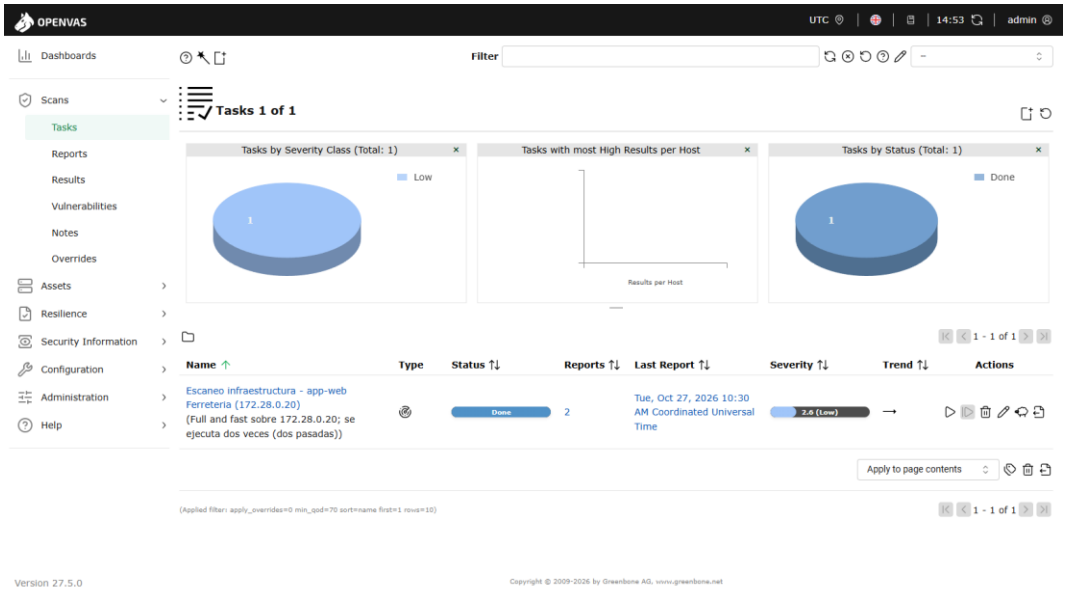


Figura 55. Tarea con dos reportes (dos pasadas), estado Done y severidad 2,6 (Baja).

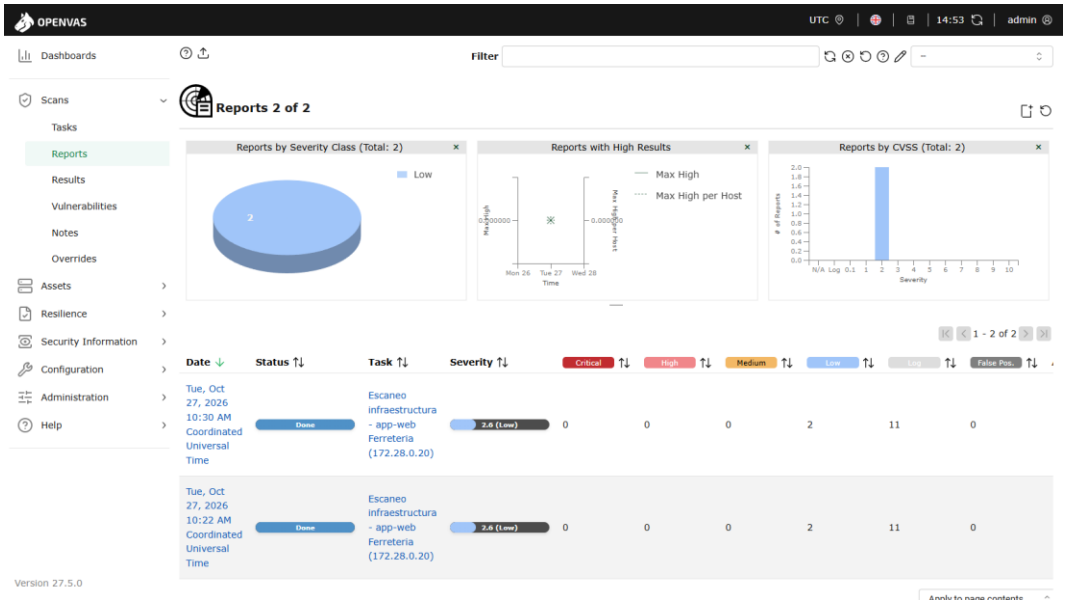


Figura 56. Reportes de ambas pasadas con el conteo por severidad.

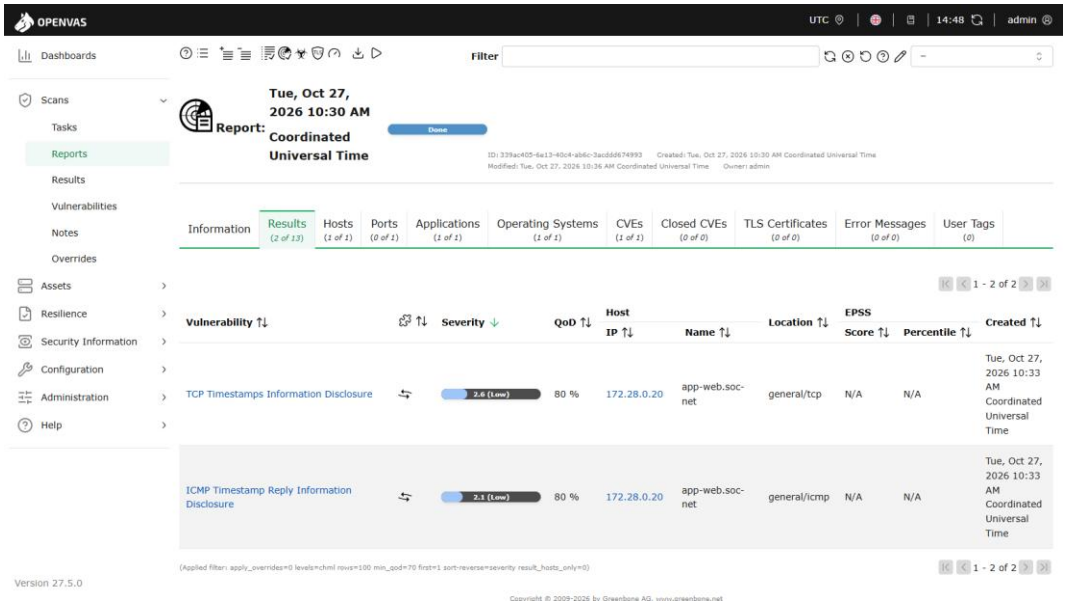


Figura 57. Pasada 2: mismos hallazgos de severidad Baja.

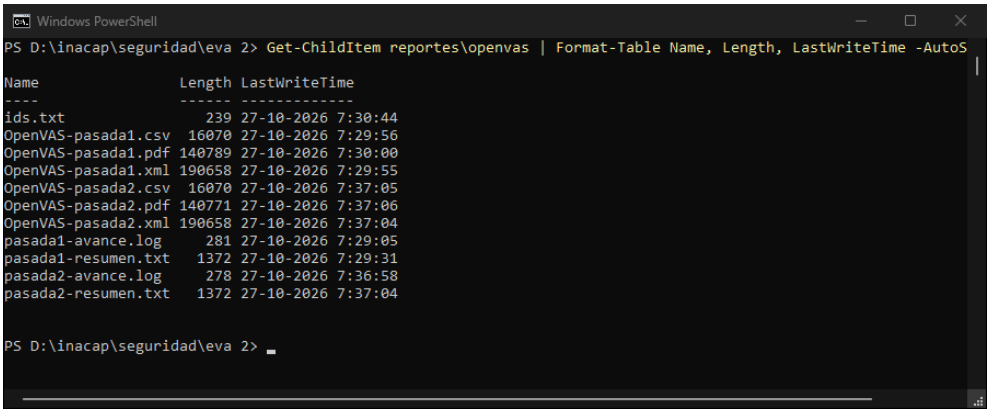


Figura 58. Reportes de OpenVAS exportados (XML, CSV y PDF) por pasada.

Tabla 6

Comparación de las dos pasadas de OpenVAS

Hallazgo (NVT)	CVSS	Pasada 1	Pasada 2	Riesgo
TCP Timestamps Information Disclosure	2,6	Sí	Sí	**Bajo**
ICMP Timestamp Reply Information Disclosure	2,1	Sí	Sí	**Bajo**
Gunicorn Detection (HTTP) / HTTP Server Banner	0,0	Sí	Sí	**Informativo**
Otros registros (detección de SO, servicios, cabeceras, CPE, traceroute)	0,0	Sí (9)	Sí (9)	**Informativo**

Nota. Pasada 1: 07:23–07:29; pasada 2: 07:31–07:36 (hora de Chile). Fuente: OpenVAS-pasada1.xml y OpenVAS-pasada2.xml.

6.5 Revisión de configuración

Además de los escáneres, se revisaron los archivos Compose, los registros y las versiones utilizadas. Esta revisión encontró la mala práctica que el docente dejó a propósito —las credenciales de Oracle en texto plano—, que ninguno de los dos escáneres puede detectar porque no analizan archivos de configuración, y otros hallazgos que se incorporan a la tabla de resumen: el puerto 1521 publicado en todas las interfaces del equipo, la contraseña débil y repetida, las imágenes sin versión fija (`latest`), la clave de la API de ZAP visible en `docker logs zap`, la interfaz de OpenVAS sin cifrar y el driver `oracledb` 26.0.1, que está fuera del rango soportado oficialmente por Django 5.2 (2.3.0 a 4.0.2) (Django Software Foundation, s.f.-a).

7. Evaluación y priorización: tabla de resumen de vulnerabilidades

La Tabla 7 consolida los hallazgos únicos. Para ZAP y OpenVAS el riesgo corresponde a la categoría que asigna cada herramienta, como indicó el docente; para los hallazgos de la revisión de configuración el grupo asignó la categoría considerando el impacto sobre la aplicación. Los colores siguen el formato del enunciado.

Tabla 7

Resumen de vulnerabilidades encontradas

ID	Vulnerabilidad	Fuente	Activo	Impacto	Riesgo
V01	Credenciales de Oracle en texto plano en el docker-compose.yml (ORACLE_PASSWORD, APP_USER_PASSWORD)	Revisión de configuración	oracle-db (compose)	Quien acceda al archivo obtiene acceso total a la base de datos (SYS y SYSTEM)	**Alto**
V02	Contraseña débil y repetida (MyPass123) para SYS, SYSTEM y el usuario de aplicación	Revisión de configuración	oracle-db	Fácil de adivinar; si se compromete una cuenta, se comprometen todas	**Alto**
V03	Puerto 1521 de Oracle publicado en todas las interfaces del equipo (0.0.0.0:1521)	Revisión (docker ps)	oracle-db	La base de datos queda expuesta a la red local además de soc-net	**Medio**
V04	Sitio servido solo por HTTP, sin HTTPS	ZAP esc. 2 (10106)	app-web	Credenciales del panel y cookies viajan sin cifrar y pueden interceptarse	**Medio**
V05	Cabecera Content-Security-Policy no configurada	ZAP esc. 1 y 2 (10038)	app-web	Sin una segunda barrera ante XSS e inyección de contenido	**Medio**
V06	CORS permisivo en archivos estáticos (Access-Control-Allow-Origin: *)	ZAP esc. 1 y 2 (10098)	app-web (/static/)	Cualquier sitio puede leer los estáticos; impacto real bajo porque son públicos	**Medio**
V07	Interfaz web de OpenVAS sin cifrar (HTTP)	Revisión (login GSA)	openvas	La contraseña del administrador del escáner viaja en claro	**Medio**
V08	Cookie csrftoken sin atributo HttpOnly	ZAP (10010)	app-web	JavaScript puede leer el token CSRF; Django indica que no aporta protección práctica	**Bajo**
V09	TCP Timestamps Information Disclosure	OpenVAS pasadas 1 y 2	app-web (172.28.0.20)	Permite estimar el tiempo de actividad del sistema	**Bajo**
V10	ICMP Timestamp Reply Information Disclosure	OpenVAS pasadas 1 y 2	app-web (172.28.0.20)	Revela la hora del sistema y facilita el reconocimiento	**Bajo**
V11	Driver oracledb 26.0.1 fuera del rango soportado por Django 5.2	Revisión de dependencias	imagen ferreteria-web:1.0	Combinación no probada por Django; riesgo de fallas o parches no coordinados	**Bajo**
V12	Imágenes con etiqueta latest (oracle-free, zap-stable, openvas)	Revisión de configuración	composes 01, 03 y 04	Versiones no reproducibles; no se sabe con certeza qué versión se evaluó	**Bajo**

ID	Vulnerabilidad	Fuente	Activo	Impacto	Riesgo
V13	Clave de la API de ZAP visible en docker logs zap	Revisión (docker logs)	zap	Quien lea los registros puede controlar ZAP por su API	**Bajo**
V14	Feeds de OpenVAS sin actualizar (SKIPSYNC, feeds del 06-10-2026)	Incidente de despliegue	openvas	No se detectan vulnerabilidades publicadas después de esa fecha	**Bajo**
V15	Divulgación de la tecnología del servidor (Server: gunicorn)	OpenVAS (registro) / cabeceras	app-web	Facilita el reconocimiento; no expone la versión	**Informativo**
V16	Atributo HTML controlable por el usuario (posible XSS)	ZAP (10031)	app-web (/?q, login)	Django escapa la salida; el escaneo activo no confirmó XSS	**Informativo**
V17	Solicitud de autenticación y gestión de sesión identificadas	ZAP (10111, 10112)	app-web (/admin/login/)	Solo identifica el formulario de ingreso y las cookies de sesión	**Informativo**
V18	User Agent Fuzzer	ZAP (10104)	app-web	Respuestas distintas según el User-Agent; sin impacto	**Informativo**

Nota. Riesgo de ZAP y OpenVAS según la herramienta; riesgo de la revisión de configuración asignado por el grupo.

Tabla 8

Cantidad de hallazgos por nivel de riesgo

Riesgo	Cantidad	Hallazgos
Alto	2	V01, V02
Medio	5	V03 a V07
Bajo	7	V08 a V14
Informativo	4	V15 a V18

8. Actuar: estrategia de mitigación

La estrategia ordena los hallazgos por prioridad —primero los de mayor riesgo y luego los de menor esfuerzo— y define para cada uno el tratamiento que propone el ciclo: **corregir** la causa, **mitigar** reduciendo su impacto o probabilidad, o **aceptar** el riesgo cuando es muy bajo o no se puede hacer nada razonable. La última columna indica cómo se verificaría la corrección.

Tabla 9
Estrategia de mitigación priorizada

Prior.	ID	Tratamiento	Acción propuesta	Verificación
1	V01	Corregir	Mover las contraseñas a un archivo .env (env_file) o a Docker secrets, excluirlo del repositorio y cambiarlas	docker compose config sin contraseñas visibles; revisión del repositorio
2	V02	Corregir	Contraseñas largas y distintas por cuenta (resetPassword); usuario de aplicación con mínimos privilegios	Intento de ingreso con la clave antigua (debe fallar)
3	V03	Corregir	Quitar la publicación del puerto (Oracle solo en soc-net) o limitarla a 127.0.0.1:1521	docker ps sin 0.0.0.0:1521; escaneo de puertos desde otro equipo
4	V04	Corregir	Proxy inverso con TLS (nginx o Traefik); activar SECURE_SSL_REDIRECT, cookies Secure y HSTS en Django	Nuevo escaneo activo de ZAP sin «HTTP Only Site»
5	V05	Corregir (aplicado en 1.1)	Middleware que envía Content-Security-Policy (default-src 'self', frame-ancestors 'none')	Escaneo de verificación: alerta resuelta (sección 9)
6	V06	Corregir (aplicado en 1.1)	WHITENOISE_ALLOW_ALL_ORIGINS = False	Escaneo de verificación: alerta resuelta (sección 9)
7	V07	Corregir	Activar HTTPS en GSA (variable HTTPS de la imagen) y publicar 9392 solo en 127.0.0.1	Ingreso sin la advertencia «Connection unencrypted»
8	V11	Corregir (aplicado en 1.1)	Fijar oracledb==4.0.2 y reconstruir la imagen	pip en el contenedor muestra 4.0.2; aplicación healthy
9	V12	Corregir	Fijar versiones o digest de las imágenes y revisarlas cada mes	docker compose config / docker images
10	V14	Mitigar	Programar la sincronización de feeds fuera de horario (SKIPSYNC=false) o actualizar la imagen cada semana	Fecha de los feeds en Administration → Feed Status
11	V13	Mitigar	Cambiar la clave, restringir el acceso a los registros y pasar la clave por archivo de configuración	docker logs zap sin la clave
12	V08	Aceptar	Riesgo residual documentado: según Django, marcar la cookie CSRF como HttpOnly no aporta protección práctica; la cookie de sesión sí es HttpOnly	Revisión de Set-Cookie de sessionid (HttpOnly)
13	V09, V10	Aceptar / mitigar	Red interna aislada; si se requiere, desactivar net.ipv4.tcp_timestamps y filtrar ICMP timestamp en el firewall del host	Nueva pasada de OpenVAS

Prior.	ID	Tratamiento	Acción propuesta	Verificación
14	V15 a V18	Aceptar	Hallazgos informativos sin impacto directo; se mantienen en observación	Siguiente ciclo de escaneos

9. Verificar y mejorar

9.1 Aplicación de correcciones y escaneo de verificación

Para cerrar una vuelta del ciclo se aplicaron las tres correcciones de menor esfuerzo sobre la aplicación, sin tocar el compose de Oracle del docente: un *middleware* que agrega la cabecera Content-Security-Policy (V05), la configuración `WHITENOISE_ALLOW_ALL_ORIGINS = False` (V06) y el cambio a `oracledb==4.0.2` (V11). Se construyó la versión `ferreteria-web:1.1` en 27 segundos, reutilizando las capas en caché (Figura 59), y se redesplegó; las cabeceras confirmaron la CSP y la ausencia de `Access-Control-Allow-Origin` en los estáticos (Figura 60).

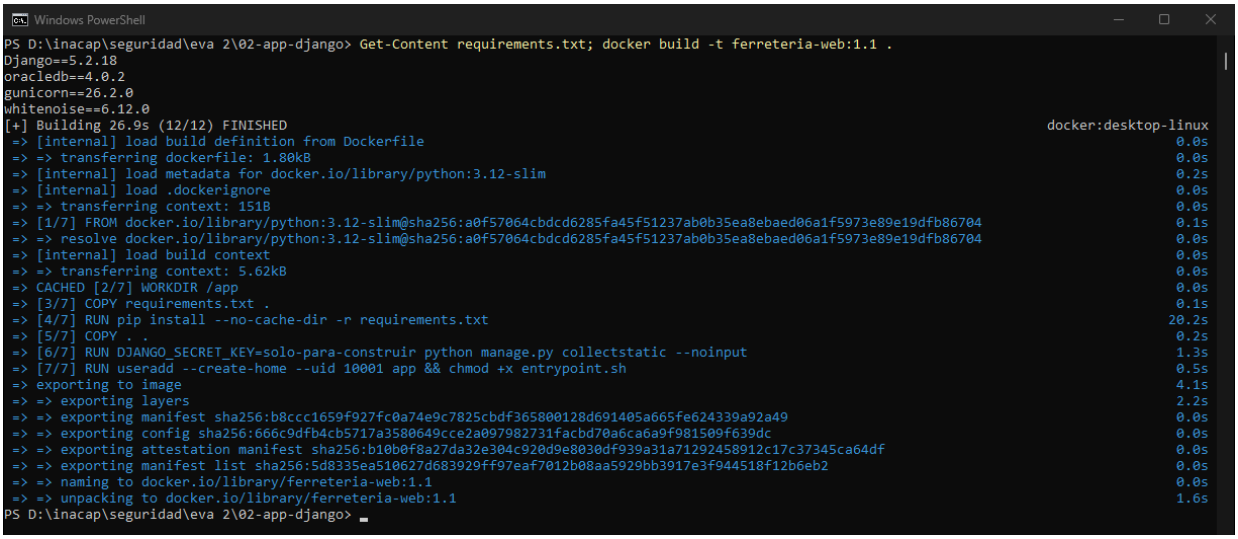


Figura 59. Construcción de la versión 1.1 con oracledb 4.0.2.

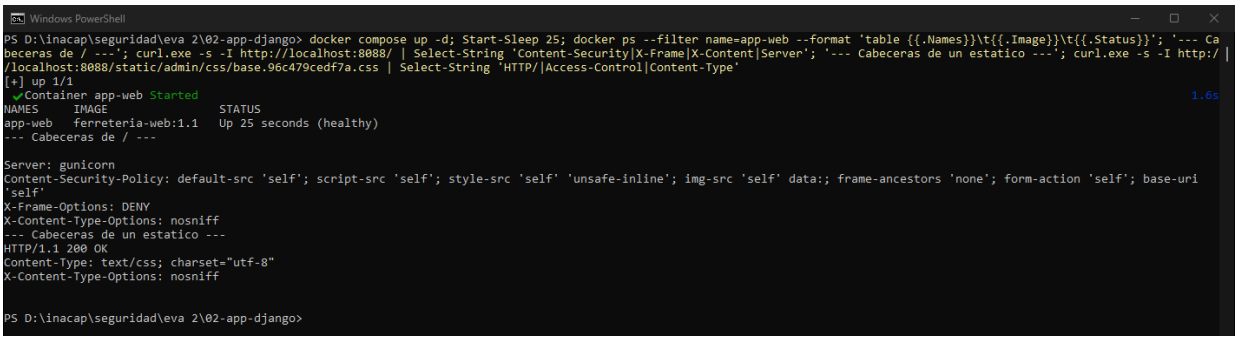


Figura 60. Aplicación 1.1 en ejecución: CSP presente y estáticos sin CORS permisivo.

Luego se ejecutó un escaneo de verificación con ZAP en una sesión nueva (Figuras 61 y 62). Las alertas «CSP Header Not Set» y «Cross-Domain Misconfiguration» desaparecieron. Apareció una alerta nueva de riesgo Medio, «CSP: style-src unsafe-inline», porque la política permite estilos en línea que usa el panel de administración de Django. Este resultado muestra por qué el proceso es cíclico: cada corrección se verifica y puede dejar un punto a mejorar en la siguiente vuelta, en este caso reemplazar los estilos en línea por archivos CSS o usar *nonces* para quitar `'unsafe-inline'`.

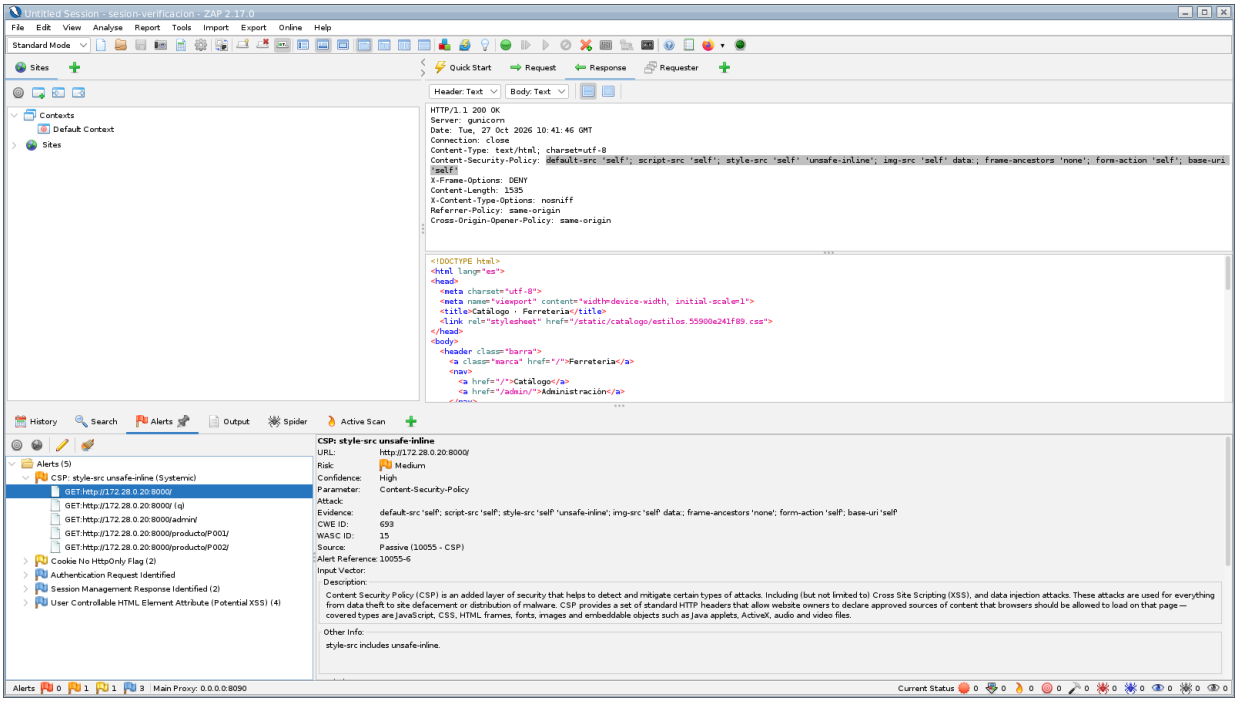
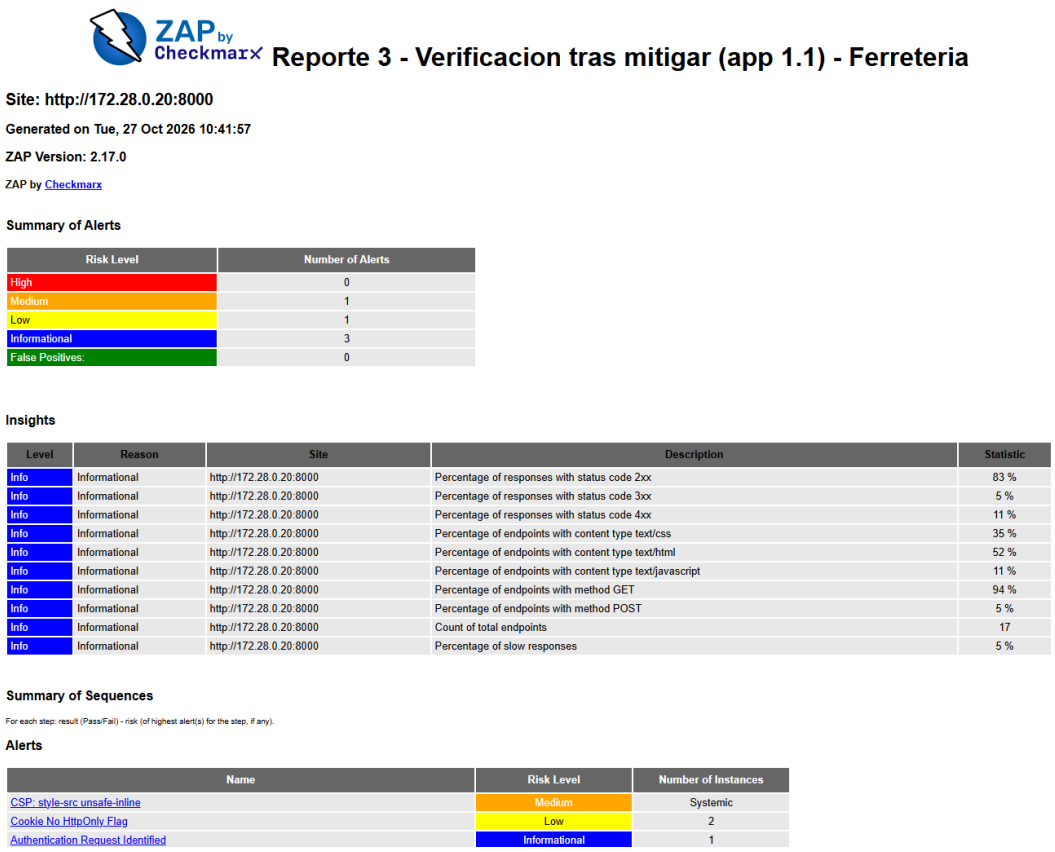


Figura 61. Escaneo de verificación: cabecera CSP en la respuesta y alertas restantes.



ZAP by Checkmarx Reporte 3 - Verificacion tras mitigar (app 1.1) - Ferreteria

Site: http://172.28.0.20:8000

Generated on Tue, 27 Oct 2026 10:41:57

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	1
Informational	3
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://172.28.0.20:8000	Percentage of responses with status code 2xx	83 %
Info	Informational	http://172.28.0.20:8000	Percentage of responses with status code 3xx	5 %
Info	Informational	http://172.28.0.20:8000	Percentage of responses with status code 4xx	11 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with content type text/css	35 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with content type text/html	52 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with content type text/javascript	11 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with method GET	94 %
Info	Informational	http://172.28.0.20:8000	Percentage of endpoints with method POST	5 %
Info	Informational	http://172.28.0.20:8000	Count of total endpoints	17
Info	Informational	http://172.28.0.20:8000	Percentage of slow responses	5 %

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
CSP: style-src unsafe-inline	Medium	Systemic
Cookie No HttpOnly Flag	Low	2
Authentication Request Identified	Informational	1

Figura 62. Reporte 3 de ZAP (verificación tras mitigar).

Tabla 10

Comparación antes y después de las correcciones

Hallazgo	App 1.0 (escaneos 1 y 2)	App 1.1 (verificación)	Estado
V05 Content-Security-Policy no configurada	Presente (Medio)	No aparece	Corregido
V06 CORS permisivo en estáticos	Presente (Medio)	No aparece	Corregido
V11 oracledb fuera del rango soportado	26.0.1	4.0.2	Corregido
CSP: style-src unsafe-inline (10055)	—	Presente (Medio)	Nuevo: siguiente iteración
V08 Cookie csrftoken sin HttpOnly	Presente (Bajo)	Presente (Bajo)	Aceptado

9.2 Métricas

Tabla 11

Métricas de los escaneos realizados

Escaneo	Herramienta	Hora (Chile)	Alcance	Alto	Medio	Bajo	Info.
Escaneo 1 (pasivo)	ZAP	07:14–07:15	16 URL	0	2	1	3
Escaneo 2 (activo)	ZAP	07:17–07:22	3 444 peticiones	0	3	1	4

Escaneo	Herramienta	Hora (Chile)	Alcance	Alto	Medio	Bajo	Info.
Pasada 1	OpenVAS	07:23–07:29	1 host, TCP IANA	0	0	2	11
Pasada 2	OpenVAS	07:31–07:36	1 host, TCP IANA	0	0	2	11
Verificación (app 1.1)	ZAP	07:41–07:42	16 URL	0	1	1	3

Nota. Cantidad de alertas o hallazgos únicos. En OpenVAS, «Info.» corresponde a registros (Log) sin severidad.

9.3 Mejora continua

Para que el ciclo no dependa de una revisión manual se propone: ejecutar una pasada de OpenVAS cada semana con un *schedule* de GSA; incorporar el escaneo base de ZAP (`zap-baseline.py`) cada vez que se construya una nueva versión de la imagen; revisar mensualmente las versiones de las imágenes base y de las dependencias de `requirements.txt`; mantener actualizados los feeds de OpenVAS; y registrar en cada vuelta las métricas de la Tabla 11 para observar su tendencia.

10. Conclusiones

El trabajo permitió recorrer una vuelta completa del ciclo de gestión de vulnerabilidades sobre una infraestructura real de contenedores. Se construyó una aplicación propia, se empaquetó con un Dockerfile, se desplegó junto a Oracle y a dos escáneres en una red aislada, y se identificaron, evaluaron, priorizaron y trataron sus vulnerabilidades. Lo más valioso no fue la cantidad de hallazgos, sino haber documentado un proceso ordenado y repetible: cualquier integrante puede reconstruir la infraestructura a partir de los cuatro archivos Compose y del Dockerfile, sin depender de comandos recordados de memoria.

Los escáneres confirmaron que la aplicación no presenta vulnerabilidades críticas ni altas: el escaneo activo de ZAP no logró inyecciones SQL ni XSS, gracias al uso del ORM y del escape automático de plantillas de Django y a su protección CSRF. Los hallazgos de ZAP se concentraron en la configuración del servidor web (cabeceras, HTTPS y CORS) y los de OpenVAS fueron de severidad baja, coherentes entre las dos pasadas. Esto refleja que los marcos de desarrollo modernos protegen bien contra los ataques clásicos, pero no reemplazan una configuración segura del despliegue.

Los riesgos más altos no fueron detectados por los escáneres, sino por la revisión de configuración: las credenciales de Oracle en texto plano y la contraseña débil repetida. Esto demuestra que las herramientas automáticas son necesarias pero no suficientes; ZAP y OpenVAS analizan servicios en ejecución y no leen archivos de configuración. Un proceso de gestión de vulnerabilidades debe combinar escaneos con revisión manual de la infraestructura como código.

Los dos incidentes de despliegue —el usuario de Oracle que no se creó y los reinicios en bucle de OpenVAS— mostraron la importancia de leer los registros de los contenedores y de entender las imágenes que se usan. En ambos casos la causa estaba documentada en los registros

y la solución se aplicó sin improvisar. Del mismo modo, la verificación de las versiones con la documentación oficial reveló que el driver de Oracle estaba fuera del rango soportado por Django, un hallazgo que ningún escáner habría reportado.

La fase de verificación confirmó el carácter cíclico del proceso: tras aplicar tres correcciones y volver a escanear, dos alertas desaparecieron y apareció una nueva derivada de la propia corrección. Que hoy una aplicación no tenga vulnerabilidades no garantiza que no las tenga mañana, cuando se publiquen nuevas fallas en Python, Django u Oracle. Por eso se propuso automatizar escaneos periódicos y revisar las versiones en cada vuelta.

Finalmente, Docker demostró ser una base adecuada para este tipo de trabajo: permitió levantar en minutos herramientas complejas como OpenVAS o ZAP sin instalarlas en el equipo, aislar los servicios en una red propia y repetir el despliegue de forma idéntica. Como propuesta de profundización queda implementar HTTPS con un proxy inverso, mover los secretos a Docker secrets y desplegar la solución en un proveedor en la nube, como mencionó el docente.

Referencias

CrowdStrike. (s.f.). *¿Qué es el ciclo de vida de la gestión de vulnerabilidades?* Recuperado el 27 de octubre de 2026, de https://www.crowdstrike.com/es-es/cybersecurity-101/exposure-management/vulnerability-management-lifecycle/

Django Software Foundation. (s.f.-a). *Databases* (Django 5.2). Recuperado el 27 de octubre de 2026, de https://docs.djangoproject.com/en/5.2/ref/databases/

Django Software Foundation. (s.f.-b). *Settings: CSRF_COOKIE_HTTPONLY* (Django 5.2). Recuperado el 27 de octubre de 2026, de https://docs.djangoproject.com/en/5.2/ref/settings/

Docker Inc. (s.f.-a). *Docker overview*. Recuperado el 27 de octubre de 2026, de https://docs.docker.com/get-started/docker-overview/

Docker Inc. (s.f.-b). *Networking overview*. Recuperado el 27 de octubre de 2026, de https://docs.docker.com/engine/network/

Docker Inc. (s.f.-c). *Volumes*. Recuperado el 27 de octubre de 2026, de https://docs.docker.com/engine/storage/volumes/

Docker Inc. (s.f.-d). *Dockerfile reference*. Recuperado el 27 de octubre de 2026, de https://docs.docker.com/reference/dockerfile/

Docker Inc. (s.f.-e). *Docker Compose*. Recuperado el 27 de octubre de 2026, de https://docs.docker.com/compose/

Greenbone AG. (s.f.). *Greenbone Community Documentation*. Recuperado el 27 de octubre de 2026, de https://greenbone.github.io/docs/latest/

Immauss. (s.f.). *OpenVAS docker container*. Recuperado el 27 de octubre de 2026, de https://immauss.github.io/openvas/

Oracle. (s.f.). *Initializing python-oracledb*. Recuperado el 27 de octubre de 2026, de https://python-oracledb.readthedocs.io/en/latest/user_guide/initialization.html

Souppaya, M. y Scarfone, K. (2022). *Guide to enterprise patch management planning: Preventive maintenance for technology* (NIST Special Publication 800-40 Rev. 4). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-40r4

ZAP. (s.f.-a). *ZAP – Docker user guide*. Recuperado el 27 de octubre de 2026, de https://www.zaproxy.org/docs/docker/about/

ZAP. (s.f.-b). *ZAP – Webswing*. Recuperado el 27 de octubre de 2026, de https://www.zaproxy.org/docs/docker/webswing/

ZAP. (s.f.-c). *Content Security Policy (CSP) Header Not Set* (alerta 10038). Recuperado el 27 de octubre de 2026, de https://www.zaproxy.org/docs/alerts/10038/

Anexos

Anexo A. Archivos docker-compose.yml

A.1 Oracle (01-oracle/docker-compose.yml, entregado por el docente)

```
version: '3.8'
services:
  oracle-db:
    image: gvenzl/oracle-free:latest
    container_name: oracle-db
    restart: unless-stopped
    ports:
      - "1521:1521"
    networks: [soc-net]
    environment:
      ORACLE_PASSWORD: MyPass123       # SYS y SYSTEM
      APP_USER: usuario                # usuario de aplicación
      APP_USER_PASSWORD: MyPass123 # contraseña del usuario
      ORACLE_DATABASE: FREEPDB1        # PDB a la que se conecta APP_USER
    volumes:
      - oracle-volume:/opt/oracle/oradata
volumes:
  oracle-volume:
    driver: local
networks:
  soc-net:
    external: true
```

A.2 Aplicación web (02-app-django/docker-compose.yml, versión final con la imagen 1.1)

```
# Aplicación web Ferretería (Django) — imagen propia construida con el Dockerfile de esta
carpeta.
# Requisitos: red soc-net creada, Oracle levantado (01-oracle) e imagen construida con:
#   docker build -t ferreteria-web:1.1 .
# Levantar desde esta carpeta:  docker compose up -d     → http://localhost:8088
services:
  app-web:
    image: ferreteria-web:1.1           # imagen local (no existe en Docker Hub); 1.1 =
mitigaciones
    container_name: app-web
    restart: unless-stopped
    ports:
      - "8088:8000"                     # host:contenedor (gunicorn escucha en 8000)
    env_file: .env                      # secretos y conexión a Oracle fuera del compose
    networks:
      soc-net:
        ipv4_address: 172.28.0.20       # IP fija: objetivo estable para ZAP y OpenVAS

networks:
  soc-net:
    external: true
```

A.3 ZAP (03-zap/docker-compose.yml)

```
# ZAP (Zed Attack Proxy) 2.17 — escáner de vulnerabilidades de aplicaciones web.
# Se usa con su interfaz gráfica completa desde el navegador (Webswing), sin instalar
nada en el equipo.
# Interfaz: http://localhost:8080/zap/   ·   Proxy/API: localhost:8090 (API protegida con
clave, ver .env)
```

```
# Levantar desde esta carpeta:  docker compose up -d
services:
  zap:
    image: zaproxy/zap-stable:latest
    container_name: zap
    restart: unless-stopped
    user: zap                           # usuario sin privilegios de la imagen oficial
    command: zap-webswing.sh            # inicia ZAP con interfaz web en lugar de
escritorio
    ports:
      - "8080:8080"                     # interfaz web (Webswing)
      - "8090:8090"                     # proxy y API de ZAP
    environment:
      # Opciones de arranque de ZAP: escuchar en todas las interfaces del contenedor,
sesión nueva
      # (evita el diálogo de persistencia) y API con clave, aceptando clientes de la red
Docker.
      ZAP_WEBSWING_OPTS: >-
        -host 0.0.0.0 -port 8090 -newsession /home/zap/sesion-eva2
        -config api.key=${ZAP_API_KEY}
        -config api.addrs.addr.name=.* -config api.addrs.addr.regex=true
    volumes:
      - ../reportes/zap:/zap/wrk        # los reportes que genera ZAP quedan en la
carpeta reportes/zap
    networks:
      - soc-net

networks:
  soc-net:
    external: true
```

A.4 OpenVAS (04-openvas/docker-compose.yml, versión final con SKIPSYNC)

```
# OpenVAS (Greenbone Community Edition) — escáner de vulnerabilidades de infraestructura.
# Imagen todo-en-uno de immauss: incluye el escáner, el gestor (gvmd) y la interfaz web
(GSA).
# Interfaz: http://localhost:9392  (usuario: admin, contraseña: la definida en .env)
# Levantar desde esta carpeta:  docker compose up -d
services:
  openvas:
    image: immauss/openvas:latest
    container_name: openvas
    restart: unless-stopped
    ports:
      - "9392:9392"                     # interfaz web GSA
    environment:
      PASSWORD: ${OPENVAS_PASSWORD}     # contraseña del usuario admin, leída desde el
archivo .env
      # La descarga de feeds desde el servidor público de Greenbone (rsync) se cortaba y
el
      # contenedor se reiniciaba en bucle. Se omite y se usan los feeds incluidos en la
imagen.
      SKIPSYNC: "true"
    volumes:
      - openvas-data:/data              # persiste la base de datos, los feeds y los
resultados
    networks:
      - soc-net

volumes:
  openvas-data:

networks:
  soc-net:
    external: true                      # la red se creó antes con: docker network create
--subnet 172.28.0.0/16 soc-net
```

Anexo B. Dockerfile de la aplicación web

```
# Imagen base oficial de Python 3.12 en su variante "slim" (Debian reducido, más
liviana).
FROM python:3.12-slim

# No generar archivos .pyc y mostrar los logs de Python al instante (sin búfer).
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# Directorio de trabajo dentro de la imagen; los comandos siguientes se ejecutan aquí.
WORKDIR /app

# Se copian primero solo las dependencias: si no cambian, Docker reutiliza esta capa en
cache.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Se copia el código de la aplicación (lo listado en .dockerignore, como .env, queda
fuera).
COPY . .

# Reúne los archivos estáticos (CSS, admin) en /app/staticfiles para que WhiteNoise los
sirva.
# La clave usada aquí solo existe durante la construcción; la real llega por variable de
entorno.
RUN DJANGO_SECRET_KEY=solo-para-construir python manage.py collectstatic --noinput

# Usuario sin privilegios: la aplicación no se ejecuta como root dentro del contenedor.
RUN useradd --create-home --uid 10001 app && chmod +x entrypoint.sh
USER app

# Documenta el puerto en que escucha gunicorn dentro del contenedor (no lo publica).
EXPOSE 8000

# Docker revisa cada 30 s que la aplicación responda en /salud/.
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s --retries=3 \
  CMD python -c "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/salud/', timeout=4)" || exit 1

# entrypoint.sh espera a Oracle y aplica las migraciones; luego ejecuta el CMD.
ENTRYPOINT ["./entrypoint.sh"]

# Servidor de producción gunicorn con 3 procesos, escuchando en todas las interfaces del
contenedor.
CMD ["gunicorn", "ferreteria.wsgi:application", "--bind", "0.0.0.0:8000", "--workers",
"3", "--access-logfile", "-"]
```

Anexo C. Comandos de la demostración

```
# Contenedores en ejecución
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"

# Contenedores de soc-net con su IP (comando del docente; en PowerShell escapar las
comillas internas)
docker network inspect soc-net -f '{{range .Containers}}{{.Name}} -
{{.IPv4Address}}{{"\n"}}{{end}}'
docker network inspect soc-net -f '{{range .Containers}}{{.Name}} -
{{.IPv4Address}}{{\"\n\"}}{{end}}'

# Interfaces web
http://localhost:8088        aplicación web (Django)
http://localhost:8080/zap/   ZAP
http://localhost:9392        OpenVAS (GSA)
```